MVST User Manual

Companion MATLAB Toolbox for
Linear Hermitian Matrix-Valued Dynamical Systems

Corentin Briat

October 2026

Contents

Preface **4**
What the toolbox is . . . 4
Mathematical foundation in one page . . . 4
Manual map . . . 5
Conventions used in this manual . . . 6

1 Getting started **7**
1.1 Installation . . . 7
1.2 The solver doctor . . . 8
1.3 External solvers . . . 9
1.4 Running the test battery . . . 9

2 Systems **11**
2.1 Mathematical setting . . . 11
2.2 The `mvst.System` factories . . . 14
2.3 Adopting legacy structs: `fromstruct` . . . 15
2.4 Solver options . . . 16
2.5 Error-identifier conventions . . . 16
2.6 Method sugar . . . 16
2.7 Worked example: a covariance model is the second moment . . . 17

3 Stability **18**
3.1 Mathematical setting . . . 18
3.2 Exact analysis: `S.stability` . . . 20
3.3 Case study: stability versus dwell-time . . . 21
3.4 Dwell-time SOS analysis: `mvst_stab_sos` . . . 23
3.5 Further stability engines . . . 23

4 Stabilization **25**
4.1 Mathematical setting . . . 25
4.2 Exact synthesis: `S.stabilize` . . . 27
4.3 Case study: the mean-square stabilizability limit . . . 27
4.4 SOS synthesis: `mvst_stabz_sos` . . . 28
4.5 Aperiodic sampled-data refresh gains . . . 29

5 Optimal control **30**
5.1 Mathematical setting . . . 30
5.2 The three horizons . . . 31
5.3 Worked case study: horizon convergence and the closed loop . . . 32
5.4 SOS layer: `mvst_oc_sos` . . . 33
5.5 Dwell-time optimal control . . . 33

6 Steering **35**
6.1 Mathematical setting . . . 35
6.2 `S.steer` . . . 36
6.3 Worked case study: an anisotropic target and the covariance trajectory . . . 37
6.4 Other classes and hard targets . . . 37

7 Dissipativity **39**
7.1 Mathematical setting . . . 39
7.2 Supply-rate convention in the toolbox . . . 40
7.3 Exact L2-gain: `S.l2gain` . . . 41
7.4 Passivity by frequency sweep: `mvst_passivity_ct` . . . 41
7.5 Dissipativity SOS: `mvst_diss_sos` . . . 41
7.6 Worked case study: compute an L2 gain exactly, then certify it by SOS . . . 42
7.7 Further dissipativity engines . . . 43

8 Games **45**
8.1 Mathematical setting . . . 45
8.2 The bounded-real interpretation, in the toolbox . . . 46
8.3 Exact level: `S.gamegamma` . . . 46
8.4 SOS relaxation: `mvst_game_sos` . . . 46
8.5 Worked case study: the critical level and the bounded-real crossover . . . 47
8.6 Further game engines . . . 48

9 Dissipative synthesis **50**
9.1 Mathematical setting . . . 50
9.2 `S.dissyn`: dissipative state feedback . . . 51
9.3 Worked case study: synthesize, inspect the gate, close the loop, re-analyze . . . 52
9.4 `mvst_dof_sos`: dynamic output feedback . . . 53

10 Positive systems **54**
10.1 The mathematical setting . . . 54
10.2 Stability: `mvst_pos_stability` . . . 55
10.3 Dwell-time stability . . . 55
10.4 Induced gains: `mvst_pos_gain` . . . 57
10.5 Dissipativity with a general linear supply: `mvst_pos_supply` . . . 58
10.6 Positivity-preserving stabilization: `mvst_pos_stabilize` . . . 58
10.7 Worked case study: an epidemic-like positive system . . . 59
10.8 Multi-mode systems and the object bridge . . . 60
10.9 Further positive-systems engines . . . 60

11 Further performance measures **63**
11.1 The performance measures . . . 63
11.2 Dwell-time and peak-to-peak measures . . . 65
11.3 Worked case study: a performance sweep . . . 65

12 Book reproduction **67**
12.1 The figure and data pipeline . . . 67
12.2 Figures and the export layer . . . 68
12.3 The runners . . . 68
12.4 The golden regression harness (development tree only) . . . 69

13 API reference **70**
13.1 How to read this reference 70
13.2 Result-struct conventions 77
13.3 Error-id namespace 77
13.4 Troubleshooting 78

Preface

The Matrix-Valued Systems Toolbox (MVST) is the companion MATLAB toolbox for the monograph ***Linear Hermitian Matrix-Valued Dynamical Systems*** by Corentin Briat. It is a reusable solver library for the analysis and synthesis of linear matrix-valued (impulsive and stochastic) dynamical systems, and it reproduces every numerical example and figure of the book.

This manual is written in author order: it starts from installation, then building a system, then walks through the analysis and synthesis problems in the same sequence the monograph presents them (stability, stabilization, optimal control, steering, dissipativity, games, synthesis, positive systems, further performance measures), and closes with book reproduction and a full API reference.

Every code snippet in this manual has been run in MATLAB (R2025a) and is shown with its actual output. No numbers are invented. (Solver diagnostics are suppressed in the displayed outputs.)

What the toolbox is

The toolbox handles five system classes: the continuous-time diffusion, the Markov-jump linear system (MJLS), the sampled-data / impulsive system, the Poissonian-jump diffusion, and the dwell-time switched system. For each class it provides:

- **exact** engines (closed-form, Riccati, or spectral): stability, stabilization, L2-gain, games, optimal control over three horizons, covariance steering, and further performance measures;
- **sum-of-squares (SOS)** relaxations for the dwell-time-constrained problems (stability, stabilization, dissipativity analysis and synthesis, games, optimal control, output feedback), which need an SDP backend (or the `nosdp` option, see Getting started);
- **copositive linear programs** for linear positive systems, which need the Optimization Toolbox.

The release-facing surface is the `+mvst` package (`mvst.System`, `mvst.Options`, `mvst.solvercheck`). Every legacy engine remains available as a `mvst_*` function in `lib/`.

Mathematical foundation in one page

So that the manual is self-contained without the monograph, here is the one idea every chapter rests on. The **state is a Hermitian matrix** $X \in \mathbb{H}^n$, almost always a second moment $X = \mathbb{E}[xx^*]$ of an underlying vector system, living in the positive semidefinite cone $\mathbb{H}^n_{\succeq 0}$. Its evolution is generated by a linear **flow operator** and a linear **jump operator**

$$\mathcal{F}(X) = \mathrm{He}(A_0 X) + \sum_{l=1}^{q} A_l X A_l^*, \qquad \mathcal{J}(X) = \sum_r J_r X J_r^*,$$

with $\mathrm{He}(M) = M + M^*$. The link to ordinary stochastics is exact: if $\mathrm{d}x = Ax\,\mathrm{d}t + \sum_l A_l x\,\mathrm{d}W_l$ then the covariance obeys $\dot{X} = \mathcal{F}(X)$, so questions about $\mathbb{E}\|x\|^2 = \mathrm{tr}\,X$ become linear-operator questions about X. Every analysis routine acts on the **lifted** matrix representation of these operators (or the monodromy over one dwell-time period), and decides problems by a spectrum, a Riccati equation, or a sum-of-squares certificate. The full development, with all class equations and margin conventions, opens Systems and Stability.

Manual map

Chapter	File	Topic	Book chapter
01	Getting started	one-step install (bundled solvers), `mvst_setup`, the six-row solver doctor, the `nosdp` option, tests	(front matter)
02	Systems	building models: the `mvst.System` factories, options, error ids	Hermitian Spaces, Cones, and Matrix-Valued Systems
03	Stability	exact analysis and dwell-time SOS	Stability
04	Stabilization	exact synthesis and SOS synthesis	Stabilization
05	Optimal control	finite / infinite / average horizons and the SOS layer	Linear-Cost Optimal Control
06	Steering	finite-time covariance steering	Steering
07	Dissipativity	L2-gain, passivity, dissipativity SOS	Dissipativity
08	Games	H-infinity game level, game SOS	Differential Games
09	Dissipative synthesis	dissipative state-feedback SOS	Dissipativity-Based Synthesis
10	Positive systems	the copositive-LP family	Linear Positive Systems
11	Further performance measures	H2, energy-to-peak, peak, peak-to-peak, dwell-time tubes	Further Performance Measures
12	Book reproduction	`run_all`, the runners, `MVST_FIGDIR`, the output folder	(all)
13	API reference	every public function with its help summary	(all)

Conventions used in this manual

- **Object first.** Examples build models with `mvst.System.<factory>(...)` and call the one-line methods (`S.stability()`, `S.l2gain()`, and so on). The underlying `mvst_*` engines are named where they matter and fully listed in the API reference.
- **Exact versus SOS.** "Exact" means a closed-form / Riccati / spectral engine that needs only base MATLAB. "SOS" means a sum-of-squares relaxation that requires SOSTOOLS and an SDP backend (see Getting started). Where both exist, the manual shows the SOS value next to the exact one.
- **Result structs.** Every analysis or synthesis routine returns a struct with documented fields; the manual reads the field that answers the question (`.stable`, `.gamma`, `.gammastar`, `.cost`, `.feas`, and so on).
- **Solver diagnostics elided.** SOSTOOLS and SeDuMi print per-solve diagnostics. In this manual those lines are suppressed so only the result is shown; when you run the snippet yourself you will additionally see the solver chatter.
- No em-dashes are used anywhere in this manual (commas, colons, and parentheses instead), matching the book's own convention.

Chapter 1

Getting started

1.1 Installation

The toolbox is plain MATLAB source: there is nothing to compile. The solvers it needs (SOSTOOLS, SeDuMi, SDPT3, SOSNOSDP) ship with it, each as an unmodified upstream snapshot in its own subfolder of `thirdparty/`. Copy or clone the repository, start MATLAB in the `matlab/` directory, and run the setup helper once per session:

```
mvst_setup
```

`mvst_setup` puts the following on the path:

- the `+mvst` package parent (the `matlab/` folder itself);
- the engine layer `lib/`;
- the plotting and export helpers `bookutil/` (figure style, PDF export);
- the test battery `tests/`;
- every bundled solver folder found under `thirdparty/` (`sostools`, `sedumi`, `sdpt3`, `sosnosdp`).

It then runs the solver doctor (next section). The setup prints one line per solver, for example:

```
MVST companion library added to the path.
  root : .../matlab
  lib  : .../matlab/lib
  sostools : added from .../matlab/thirdparty/sostools
  sedumi   : added from .../matlab/thirdparty/sedumi
  sdpt3    : added from .../matlab/thirdparty/sdpt3
  sosnosdp : added from .../matlab/thirdparty/sosnosdp
```

Options:

- `mvst_setup('examples', true)` also adds `examples/` to the path;
- `mvst_setup('quiet', true)` skips the doctor report (path only);
- `mvst_setup('thirdparty', false)` does not add the bundled solvers, so that your own installations are used.

1.1.1 Using your own solver installation

The bundled folders are independent and replaceable:

- A solver that is already on the MATLAB path (found through its entry point, for instance `sosprogram` or `sedumi`) is respected: `mvst_setup` prints `already on the path, bundled copy not added` and leaves the bundled copy out.

- A subfolder of `thirdparty/` can be deleted or replaced by a newer upstream snapshot; a missing subfolder is skipped silently.
- The components, versions, and licenses are listed in `THIRD-PARTY.md` and `thirdparty/VERSIONS.md`.

1.1.2 Compiling the SDP solvers once

SeDuMi runs from its MATLAB source, but fast solving needs its platform MEX files, and SDPT3 ships without any compiled MEX file. Run these once, from the solver's own folder:

```
cd thirdparty/sedumi; install_sedumi
cd ../sdpt3;          install_sdpt3
```

The doctor prints a hint (`SDPT3 found but does not solve: run install_sdpt3 once`) whenever a solver is found but fails its probe. The exact engines never need either solver.

1.1.3 MATLAB version

The floor is **R2020a**: the code uses `arguments` blocks (R2019b) and `exportgraphics` (R2020a, used for figure export). It is developed and tested on **R2025a**; older releases are not tested, and GNU Octave is not supported. The exact engines (Riccati, stability, steering, games, L2 gain) need only base MATLAB: no Control System Toolbox is required (the Lyapunov solves use `sylvester` and a Kronecker solve).

1.2 The solver doctor

`mvst.solvercheck` detects and probes the SDP / SOS toolchain. It has six rows: SeDuMi, SDPT3, MOSEK, SOSTOOLS, SOSNOSDP, and YALMIP (optional, not bundled). After `mvst_setup` on a machine with the bundled solvers (SDPT3 not yet compiled) it reports:

```
MVST solver / SOS toolchain check
------------------------------------------------------------------------
  tool       found   solves   version      location
------------------------------------------------------------------------
  SeDuMi     yes     yes      1.3.7        .../thirdparty/sedumi/sedumi.m
  SDPT3      yes     no       4.0          .../thirdparty/sdpt3/sdpt3.m
  MOSEK      no      ?        -            (not on path)
  SOSTOOLS   yes     -        4.00         .../thirdparty/sostools/sosprogram.m
  SOSNOSDP   yes     -        0.1.0        .../thirdparty/sosnosdp/bridge/
    ↪ sossolve_nosdp.m
  YALMIP     no      -        -            (not on path; optional, legacy engines
    ↪ only)
------------------------------------------------------------------------
  4 of 6 tools detected; 1 SDP solver(s) verified to solve.
  hint: SDPT3 found but does not solve: run install_sdpt3 once (compiles its MEX
    ↪ files).
  solves column:  yes = trivial 2x2 SDP solved,  no = failed,
                  ? = not probed (no SOSTOOLS),  - = not an SDP solver.
```

The columns:

- **found**: whether the tool resolves on the MATLAB path.
- **solves**: whether a trivial 2x2 SDP was actually solved by that backend. The probe is a trivial SOSTOOLS program solved by that backend (YALMIP is not used). It is `?` when SOSTOOLS is absent, `yes`/`no` once probed, and `-` for the tools that are not themselves SDP solvers (SOSTOOLS, YALMIP, SOSNOSDP).
- **version** / **location**: what was found and where.

With `mvst_setup('thirdparty', false)` and nothing else on the path, all six rows read `no`; the exact engines still work, only the SOS layer of the Stability, Stabilization, Optimal Control, Dissipativity, and Game chapters needs a backend.

1.3 External solvers

The SOS layer requires **SOSTOOLS** (the SOS front end) and one backend, selected through `mvst.Options('solver', ...)` or the `solver` field of a plain options struct:

`solver`	Backend	Needs
`'sedumi'` (default)	SeDuMi	bundled; run `install_sedumi` once for speed
`'sdpt3'`	SDPT3	bundled; run `install_sdpt3` once
`'mosek'`	MOSEK	your own MOSEK installation
`'nosdp'`	SOSNOSDP	bundled; no SDP solver at all

The `'nosdp'` option solves the SOS programs with SOSNOSDP (no semidefinite programming). It applies to the SOSTOOLS-based engines (`mvst_*_sos` and the engines built on them), which reach the backend through the single dispatcher `mvst_sossolve`. The constant-certificate LMIs of `mvst_pos_ms`, `mvst_stab_delay`, and the decay-rate branch of `mvst_perf_dwell` are also SOSTOOLS programs by default (`opts.engine = 'sostools'`), so they accept `'nosdp'` too; their former **YALMIP** formulation is kept for one release as `opts.engine = 'yalmip'` (cross-checks only; YALMIP is an optional external dependency, not bundled, and the call fails with `mvst:opts:yalmipMissing` when it is absent). `mvst_kyp` is a SOSTOOLS program by default as well. The **Optimization Toolbox** (`linprog`) is required for the positive-systems copositive LPs.

```
sys   = struct('A0',[0 1;-1 0], 'J', 0.5*eye(2));     % example of the Stability
    ↪ chapter
dwell = struct('type','periodic','Tbar',0.5,'deg',4);
o = mvst.Options('solver', 'nosdp');       % no SDP solver involved
r = mvst_stab_sos(sys, dwell, o);          % same call, other backend
fprintf('feas=%d  slack=%.3e\n', r.feas, r.slack);
```

```
feas=1  slack=5.000e-01
```

If you request a backend that is not installed, the options normalizer refuses with no silent fallback:

```
mvst_opts(struct('solver','sdpt3'));       % with SDPT3 absent
```

```
mvst:opts:solverMissing : Requested solver 'sdpt3' is not on the MATLAB path
(probe 'sdpt3' failed).  Install it or choose another solver; no silent
fallback is performed.
```

YALMIP is not bundled. If you use it in published work (legacy engine paths), please cite it (J. Lofberg, "YALMIP: A toolbox for modeling and optimization in MATLAB", CACSD 2004), as its license requests.

1.4 Running the test battery

One entry point runs every unit-test suite and prints a PASS / FAIL / SKIPPED summary:

```
cd tests; run_all_tests
```

or from the shell:

```
matlab -batch "cd tests; run_all_tests"
```

Tests that need a solver which is not on the path are not failures: they are reported as `SKIPPED` lines and counted separately. The exit code is nonzero only if a test truly fails. Without `mvst_setup` (library only) the battery therefore reports many skipped tests and no failure; after `mvst_setup` the solver-dependent tests run. A full run takes about 10 minutes; the tail of the report looks like:

```
--------------------------- summary ----------------------------
  suite                            pass   fail   status
  ------------------------------------------------------------
  tperf                               8      0   PASS
  tpos                               14      0   PASS
  tstability                         13      0   PASS
  tsystem                            31      0   PASS
  ...
  ------------------------------------------------------------
  TOTAL: <passed> passed, 0 failed, <skipped> skipped
------------------------------------------------------------------
  ALL GREEN.
```

The suites are auto-discovered `matlab.unittest` classes, so a new `t*.m` `TestCase` is picked up without editing the harness. Individual suites run with `runtests('tpos')` and so on.

Chapter 2

Systems

This chapter builds models. Everything downstream (stability, control, dissipativity, and so on) takes a system as its first argument, so this is the common entry point. The recommended way in is the `mvst.System` class. It maps directly onto the model classes of the companion monograph chapter **Hermitian Spaces, Cones, and Matrix-Valued Systems**.

The chapter opens with the mathematics the toolbox actually computes with, so that a reader without the book can follow every later chapter. It then documents the factories, options, and the legacy-struct bridge, and closes with a worked end-to-end example that checks a covariance model against a Monte-Carlo simulation.

2.1 Mathematical setting

2.1.1 The state is a Hermitian matrix

The objects MVST analyses are not vectors but **Hermitian matrices**. The state of every model is a matrix

$$X(t) \in \mathbb{H}^n, \qquad \mathbb{H}^n = \{X \in \mathbb{C}^{n\times n} : X = X^*\},$$

and, because X is almost always a covariance or second moment, it lives in the closed convex **cone** of positive semidefinite matrices

$$\mathbb{H}^n_{\succeq 0} = \{X \in \mathbb{H}^n : v^*Xv \geq 0 \text{ for all } v \in \mathbb{C}^n\}.$$

The natural pairing is the trace (Frobenius, Hilbert-Schmidt) inner product

$$\langle P, X\rangle = \operatorname{tr}(P^*X),$$

and we write the Hermitian-part operator as

$$\operatorname{He}(M) = M + M^*.$$

(This is the doubled Hermitian part, following the monograph, not the averaged $(M+M^*)/2$.)

2.1.2 Data scope: real-symmetric engines

The theory is stated on **Hermitian** matrices $X = X^*$ with complex data, and the general displays in this manual keep that notation (the conjugate-transpose *). The **exact numerical engines**, however, implement the **real-symmetric** case: real system data $A, A_l, B, \ldots$ and real symmetric $X = X^\top$. That is what every example in this manual uses, and it matches the companion monograph's own computational caveat. Concretely, the lifts (`mvst_svecbasis`, `mvst_liftsym`) build a real $n(n+1)/2$-dimensional symmetric basis and use plain transposes, and the Riccati,

steering, and game engines transpose (never conjugate) their coefficients. The convention followed throughout: where a display writes * it states the general Hermitian theory, and where it writes $^\top$ it describes exactly what the code computes on real data. Complex-Hermitian system data is **outside the numerical scope** of the exact engines.

2.1.3 The flow operator and the jump operator

A matrix-valued system is generated by two linear operators. The **flow operator** $\mathcal{F}$ drives the continuous evolution and the **jump operator** $\mathcal{J}$ acts at isolated instants. Here E is the descriptor (left) operator of the covariance dynamics, a placeholder for a descriptor/selection map; in the square case treated throughout it is the identity, $E = I_n$ (no input selection), and the operators take the completely positive (Lindblad-Kraus) form

$$\mathcal{F}(t, X) = \mathrm{He}\left(A_0(t)X\right) + \sum_{l=1}^{q} A_l(t) X A_l(t)^*,$$

$$\mathcal{J}(k, X) = \sum_{r=0}^{p} J_r(k) X J_r(k)^*.$$

Here A_0 is the drift, the A_l are the q diffusion (multiplicative-noise) channels, and the J_r are the $p+1$ jump (reset) channels. Writing $\mathrm{He}(A_0X) = A_0X + XA_0^*$ makes the flow explicit:

$$\mathcal{F}(t, X) = A_0X + XA_0^* + \sum_{l=1}^{q} A_l X A_l^*.$$

Both operators map the cone into itself, $\mathcal{F}(\cdot)$ in the tangential sense and $\mathcal{J}(\mathbb{H}^n_{\succeq 0}) \subseteq \mathbb{H}^n_{\succeq 0}$: a nonnegative covariance stays nonnegative. This cone-invariance is the structural fact behind every result in the manual.

The **adjoints** (used by all the Lyapunov and dissipativity certificates) transpose each coefficient in place:

$$\mathcal{F}^*(t, P) = \mathrm{He}\left(A_0(t)^*P\right) + \sum_{l=1}^{q} A_l(t)^* P A_l(t), \qquad \mathcal{J}^*(k, P) = \sum_{r=0}^{p} J_r(k)^* P J_r(k).$$

2.1.4 The impulsive system

The most general model flows according to $\mathcal{F}$ between an increasing sequence of jump times $t_1 < t_2 < \cdots$ (with $t_k \to \infty$) and is reset by $\mathcal{J}$ at each t_k:

$$\dot{X}(t) = \mathcal{F}(t, X(t)), \quad t \neq t_k, \qquad X(t_k^+) = \mathcal{J}(k, X(t_k^-)),$$

with $X(t_k^-)$ and $X(t_k^+)$ the left and right limits. Setting $\mathcal{J} = \mathrm{id}$ gives a purely continuous-time system; setting $\mathcal{F} = 0$ and $t_k = k$ gives a purely discrete-time system

$$X(k+1) = \mathcal{J}(k, X(k)).$$

2.1.5 Why the state is a matrix: the second-moment link

The matrix models are not an abstraction for its own sake. They are the exact **second-moment dynamics** of ordinary vector stochastic systems. Take the linear Ito stochastic differential equation

$$\mathrm{d}x = A(t)x\,\mathrm{d}t + \sum_{l=1}^{q} A_l(t)x\,\mathrm{d}W_l,$$

with independent standard Wiener processes W_l and $x \in \mathbb{C}^n$. Define the second moment (covariance) matrix

$$X(t) = \mathbb{E}\big[x(t)x(t)^*\big] \in \mathbb{H}^n_{\succeq 0}.$$

Applying the Ito product rule to xx^* and taking expectations (the martingale terms vanish, the quadratic covariation contributes $\sum_l A_l x x^* A_l^*\,\mathrm{d}t$) gives the deterministic **covariance ODE**

$$\dot{X} = AX + XA^* + \sum_{l=1}^{q} A_l X A_l^* = \mathcal{F}(X).$$

So the flow operator with drift $A_0 = A$ and channels A_l is precisely the generator of the covariance. Mean-square questions about x (does $\mathbb{E}\|x\|^2 = \operatorname{tr} X$ decay?) become linear-operator questions about X, which is what the next chapters solve. Jumps arise the same way from a reset $x(t_k^+) = Jx(t_k^-)$, whose second moment is $X^+ = JXJ^*$.

2.1.6 The class taxonomy

Each MVST factory instantiates one specialization of the impulsive system. The table below states the covariance dynamics of every class; the drift term $\mathrm{He}(AX)$ is $AX + XA^*$ throughout, sums over l run $1, \ldots, q$, and inputs are folded into the autonomous generator (set $u = Kx$ or $u = 0$).

Tag	Vector model	Covariance dynamics
`ct`	diffusion $\mathrm{d}x = Ax\,\mathrm{d}t + \sum_l A_l x\,\mathrm{d}W_l$	$\dot{X} = \mathrm{He}(AX) + \sum_l A_l X A_l^*$
`dt`	recursion $x(k{+}1) = \sum_i \nu_i A_i x(k)$	$X(k{+}1) = \sum_i A_i X A_i^*$
`mjls`	Markov modes $\sigma(t)$, generator $\Pi = (\pi_{ij})$	$\dot{X}_i = \mathrm{He}(A_i X_i) + \sum_l A_{i,l} X_i A_{i,l}^* + \sum_j \pi_{ji} X_j$
`poisson`	diffusion + Poisson resets J_r, rates λ_r	$\dot{X} = \mathrm{He}(AX) + \sum_l A_l X A_l^* + \sum_r \lambda_r \big(J_r X J_r^* - X\big)$
`levy`	Levy-driven $\mathrm{d}x = Ax\,\mathrm{d}t + \sum_l A_l x\,\mathrm{d}L_l$	$\dot{X} = \mathrm{He}(AX) + \sum_l c_l A_l X A_l^*$
`switched`	mode signal $\sigma(t)$, dwell-time schedule	$\dot{X} = \mathrm{He}(A_\sigma X) + \sum_l A_{\sigma,l} X A_{\sigma,l}^*$
`impulsive`	flow A, scheduled reset J at t_k	$\dot{X} = \mathrm{He}(AX) + \sum_l A_l X A_l^*$, $X^+ = JXJ^*$
`sampled`	plant + zero-order-hold on u, period $\bar{T}$	augmented flow with hold, jump refreshes u every $\bar{T}$

For the modal classes the second moment splits into the mode-indexed tuple $X_i = \mathbb{E}[xx^*\mathbf{1}_{\sigma=i}]$, coupled through the generator Π (continuous time) or the transition matrix P (discrete time); the true covariance is recovered as $\sum_i X_i$. In the discrete-time (`dt`) recursion $x(k{+}1) = \sum_i \nu_i A_i x(k)$ the ν_i are independent, identically distributed, zero-mean **unit-variance** scalars ($\mathbb{E}[\nu_i \nu_j] = \delta_{ij}$), which is why the second-moment map drops them and reads $X(k{+}1) = \sum_i A_i X A_i^*$. The `levy` class carries a variance-rate weight $c_l = \sigma_l^2 + \int z^2 \nu_l(\mathrm{d}z)$ per channel (with $\nu_l(\mathrm{d}z)$ the Levy measure), the Wiener case being $c_l = 1$; the engines fold this rate into an **effective channel**

$A_c = \sqrt{c_l}A_l$ (and the Levy mean drift into A_0), so that $c_l A_l X A_l^* = (\sqrt{c_l}A_l)X(\sqrt{c_l}A_l)^*$. For a Poisson jump-diffusion the reset writes $J_r - I$ inside the SDE but J_r inside the covariance term, which is why the generator carries $\lambda_r(J_r X J_r^* - X)$.

For deeper theory (well-posedness, the transition operator, complete positivity, Choi's theorem, cross-positivity) see the companion monograph. The rest of this chapter is how you build these objects in code.

2.2 The `mvst.System` factories

`mvst.System` is an immutable value object built through a static factory, one per system class of the book. Each factory validates its inputs and infers the class tag. All of them accept the shared name/value cost and noise blocks (`'Q'`, `'R'`, `'S'`, `'W'`, `'XO'`, `'Ac'`, `'Bc'`) and, where relevant, the disturbance / output blocks (`'Bw'`, `'C'`, `'Dw'`). The `'Ac'` block is the stack of diffusion channels A_l (the state-multiplicative noise) and `'Bc'` the input channels B_l.

2.2.1 Continuous-time diffusion: `ct`

The controlled diffusion $\mathrm{d}x = (Ax + Bu)\,\mathrm{d}t + \sum_l (A_l x + B_l u)\,\mathrm{d}W_l$, whose covariance obeys $\dot{X} = \mathrm{He}(AX) + \sum_l A_l X A_l^*$ once the feedback is folded in.

```
A = [-1 0.5; 0 -2];
S = mvst.System.ct(A, [1;1], 'Q', eye(2), 'R', 1);
S.n, S.m, S.tag
```

```
ans =
     2
ans =
     1
ans =
    'ct'
```

`S.n`, `S.m`, `S.M` are dependent properties: state dimension, input dimension, and number of modes. `S.tag` is the resolved class.

2.2.2 Markov-jump linear system: `mjls`

Modes switch under a continuous-time Markov generator `Pi` (Metzler, zero row sums), so the covariance is the coupled tuple $\dot{X}_i = \mathrm{He}(A_i X_i) + \sum_l A_{i,l} X_i A_{i,l}^* + \sum_j \pi_{ji} X_j$. Pass the per-mode drift and input matrices as cells.

```
Pi = [-1 1; 2 -2];
S = mvst.System.mjls({[-1 0;0 -2],[-3 1;0 -1]}, {[1;0],[0;1]}, Pi, 'Q', eye(2));
S.M, S.tag
```

```
ans =
     2
ans =
    'mjls'
```

The name/value cost and noise blocks are shared across modes. For per-mode cost data, build the modes with `mvst_mode` and wrap the assembled `mvst_system` with `mvst.System.fromstruct`.

2.2.3 Sampled-data (zero-order hold): `sampled`

A continuous-time flow with sampling period `T` and sample-and-hold reset `J`. The state is augmented to (x, u); the control is held constant along the flow ($\dot{u} = 0$) and refreshed at each sampling instant, exactly the sampled-data impulsive realization of the monograph.

```
S = mvst.System.sampled([-1 1; 0 -1], [0;1], eye(2), 0.2);
S.tag
```

```
ans =
    'sampled'
```

2.2.4 Poissonian-jump diffusion: `poisson`

A diffusion with spontaneous (Poisson) jump maps `J` (n x n x pf) at rates `lambda` (1 x pf), whose covariance carries the generator term $\sum_r \lambda_r (J_r X J_r^* - X)$.

```
S = mvst.System.poisson([-1 0;0 -1], [1;0], 0.5*eye(2), 2);
S.tag
```

```
ans =
    'poisson'
```

2.2.5 Dwell-time switched system: `switched`

Modes with no stochastic coupling (zero generator), governed by a dwell-time schedule, with flow generator $\mathrm{He}(A_\sigma X) + \sum_l A_{\sigma,l} X A_{\sigma,l}^*$ in the active mode. `'Td'` carries the dwell-time for the dwell-time backends.

```
S = mvst.System.switched({[-1 0;0 -2],[-2 1;0 -1]}, {[1;0],[0;1]}, 'Td', 0.5);
S.M, S.tag
```

```
ans =
     2
ans =
    'switched'
```

2.2.6 Impulsive system with a scheduled reset: `impulsive`

A continuous-time flow with a scheduled (dwell-time) reset map `J`: flow $\dot{X} = \mathrm{He}(AX) + \sum_l A_l X A_l^*$ between jumps, reset $X^+ = JXJ^*$ at each t_k.

```
S = mvst.System.impulsive([0 1; -1 0], [0;1], 0.5*eye(2));
S.tag
```

```
ans =
    'impulsive'
```

2.3 Adopting legacy structs: `fromstruct`

Two struct conventions coexist in the engine layer: **thin** (the OC / Riccati / steering layer, drift `A`, input `B`) and **rich** (the SOS layer, drift `A0`, input `B0`). `mvst.System.fromstruct` adopts any

of a thin bare mode (`mvst_mode` output), a thin system (`mvst_system` output), or a rich struct. For a thin input the round trip is exact:

```
md = mvst_mode([-1 0;0 -2], [1;1], eye(2), 1);
S  = mvst.System.fromstruct(md);
isequal(S.thin(), md)
```

```
ans =
  logical
   1
```

The two bridge methods `S.thin()` and `S.rich()` emit the struct each engine family consumes; you rarely call them directly, since the analysis methods do it for you.

2.4 Solver options

`mvst.Options` fixes the release-facing option surface (`solver`, `reltol`, `verbosity`) with validation, and threads the `.solver` choice through every SDP / SOS engine.

```
o = mvst.Options('solver','sedumi','verbosity',0);
o.solver, o.reltol
```

```
ans =
    'sedumi'
ans =
   1.0000e-08
```

Recognized solvers are `'sedumi'` (default), `'sdpt3'`, and `'mosek'`. A plain struct with a `.solver` field is accepted anywhere an `mvst.Options` is; the engines normalize both through `mvst_opts`.

2.5 Error-identifier conventions

All errors use the form `mvst:<name>:<reason>`, where `<name>` is the short function or class name and `<reason>` is a lowerCamel tag. The class factories use the CamelCase class name, for example a non-square drift:

```
S = mvst.System.ct([1 2 3], [1;1]);
```

```
mvst:System:notSquare : A must be square; got 1 x 3.
```

Input-validation errors are additive: they fire only on inputs that would otherwise crash cryptically, never on a currently-working call. The identifier strings are a stable contract (the tests assert them), so you can catch on `mvst:System:*`, `mvst:opts:solverMissing`, `mvst:perf:unsupported`, and the like.

2.6 Method sugar

Once built, a system exposes every analysis and synthesis routine as a one-line method that forwards to the matching engine: `l2gain`, `steer`, `gamegamma`, `ocfinite`, `ocinfinite`, `ocaverage`, `stability`, `stabilize`, `dissyn`, `pos`, `perf`. The rest of the manual uses these.

2.7 Worked example: a covariance model is the second moment

This example closes the loop on the second-moment link above. It builds the continuous-time covariance generator of a scalar-noise diffusion, propagates it, and checks the result against a direct Monte-Carlo simulation of the underlying vector SDE. Take

$$\mathrm{d}x = Ax\,\mathrm{d}t + A_c x\,\mathrm{d}W, \qquad A = \begin{bmatrix} -1 & 0.5 \\ 0 & -2 \end{bmatrix}, \quad A_c = 0.3\begin{bmatrix} 0 & 0 \\ 1 & 0 \end{bmatrix},$$

started from $x(0) \sim \mathcal{N}(0, I_2)$, so $X(0) = I_2$. The covariance obeys $\dot{X} = AX + XA^* + A_c X A_c^*$, which in vectorized form is the linear ODE $\dot{\mathrm{vec}}(X) = \left(I \otimes A + A \otimes I + A_c \otimes A_c\right)\mathrm{vec}(X)$.

```
A = [-1 0.5; 0 -2]; Ac = 0.3*[0 0;1 0]; X0 = eye(2); n = 2; tf = 2;
Kfull = kron(eye(n),A) + kron(A,eye(n)) + kron(Ac,Ac);   % covariance operator
Xode  = reshape(expm(tf*Kfull)*X0(:), n, n);             % covariance at t = 2

rng(0); N = 20000; dt = 1e-3; tt = 0:dt:tf;              % Monte-Carlo of the SDE
X = chol(X0,'lower')*randn(2,N);
for k = 1:numel(tt)-1
    X = X + (A*X)*dt + (Ac*X).*(sqrt(dt)*randn(1,N));
end
Xmc = (X*X')/N;
fprintf('covariance-ODE trace = %.4f\n', trace(Xode));
fprintf('Monte-Carlo   trace = %.4f\n', trace(Xmc));
```

```
covariance-ODE trace = 0.0233
Monte-Carlo   trace = 0.0233
```

The two agree to the printed digits, and the full matrices match as well:

```
disp(Xode); disp(Xmc);
```

```
    0.0221    0.0014
    0.0014    0.0013

    0.0220    0.0014
    0.0014    0.0013
```

The covariance ODE is exact and the Monte-Carlo estimate converges to it. This is the sense in which "the state is a matrix": the matrix generator you build with `mvst.System.ct` is the generator of the second moment of the vector SDE, and the toolbox reasons about it exactly rather than by sampling. Every analysis routine in the following chapters acts on this generator (or its monodromy over one dwell-time period), not on trajectories.

Chapter 3

Stability

This chapter covers mean-square stability analysis: the exact spectral tests of `mvst_stability` (method `S.stability`) and the dwell-time sum-of-squares tests of `mvst_stab_sos`. It corresponds to the companion monograph chapter **Stability**.

It opens with the mathematical setting (what mean-square stability means, why a spectrum decides it, how dwell-time notions and monodromy operators work, and the margin conventions the toolbox reports), documents the exact and SOS engines, and closes with an end-to-end stability-versus-dwell-time case study whose critical dwell-time is confirmed against a closed-form value.

3.1 Mathematical setting

3.1.1 What is being stabilized

Recall from Systems that the state is a Hermitian matrix $X(t)$, almost always the second moment $X = \mathbb{E}[xx^*]$ of an underlying vector system. Because $\operatorname{tr} X = \mathbb{E}\|x\|^2$, decay of X is exactly **mean-square** decay of x.

3.1.2 Mean-square exponential stability

The system is (uniformly) exponentially stable when the matrix state contracts at a geometric rate. For an impulsive system with flow rate $\alpha > 0$ and per-jump contraction $\rho \in (0, 1)$ the definition is the hybrid bound

$$\|X(t)\| \le \beta e^{-\alpha(t-s)} \rho^{\kappa(t,s)} \|X(s)\|, \qquad t \ge s \ge t_0,$$

with $\beta \ge 1$, $\kappa(t, s)$ the number of jumps in $(s, t]$, and $\|X\| = \operatorname{tr} X$. Two degenerate cases specialize it: persistent flowing ($\rho = 1$) gives $\|X(t)\| \le \beta e^{-\alpha(t-s)} \|X(s)\|$, and persistent jumping ($\alpha = 0$) gives $\|X(t)\| \le \beta \rho^{\kappa(t,s)} \|X(s)\|$.

Since $\|X\| = \operatorname{tr} X = \mathbb{E}\|x\|^2$, this is mean-square exponential stability of the vector system. For the pure continuous-time diffusion $\mathrm{d}x = A_0 x\,\mathrm{d}t + \sum_l A_l x\,\mathrm{d}W_l$ it is equivalent to the existence of $P \succ 0$ with

$$A_0^* P + P A_0 + \sum_{l=1}^{q} A_l^* P A_l \prec 0,$$

and for the discrete recursion $x(k{+}1) = \sum_i \nu_i A_i x(k)$ (the ν_i the i.i.d. unit-variance scalars of Systems) to $\sum_i A_i^* P A_i - P \prec 0$. These are the Lyapunov identities the spectral tests below decide exactly.

3.1.3 The lift, and why a spectrum decides stability

The flow and jump operators are linear on the space of Hermitian matrices, so they have a **matrix representation**. For the **real data** the engines handle (see the data-scope note in Systems), the identity $\mathrm{vec}(AXB^\top) = (B \otimes A)\,\mathrm{vec}(X)$ turns the operators $\mathcal{F}(X) = A_0X + XA_0^\top + \sum_l A_lXA_l^\top$ and $\mathcal{J}(X) = \sum_l J_lXJ_l^\top$ into the $n^2 \times n^2$ matrices

$$M_{\mathcal{F}} = I \otimes A_0 + A_0 \otimes I + \sum_l A_l \otimes A_l, \qquad M_{\mathcal{J}} = \sum_l J_l \otimes J_l,$$

with **no conjugation** (the general Hermitian form carries $\overline{A_0}$, $\overline{A_l}$, $\overline{J_l}$, which coincide with these on real data). This is exact for real data. Because the eigenmatrices are symmetric, one restricts to the real subspace through an isometry U (the scaled half-vectorization of `mvst_svecbasis`, dimension $n(n+1)/2$), giving the **reduced** real representations

$$\widetilde{M}_{\mathcal{F}} = U^\top M_{\mathcal{F}} U, \qquad \widetilde{M}_{\mathcal{J}} = U^\top M_{\mathcal{J}} U.$$

This is exactly what `mvst_liftsym` builds. The restriction is exact, so the operator spectrum equals the ordinary matrix spectrum of the reduced matrix, and stability reduces to a spectral condition:

- **Continuous-flow classes** (`ct`, `mjls`, `poisson`, `levy`): the solution of $\dot{X} = \mathcal{F}(X)$ is $e^{\widetilde{M}_{\mathcal{F}}t}\,\mathrm{svec}(X_0)$, so it decays iff the **spectral abscissa** is negative,

$$\mu(\mathcal{F}) = \max\{\mathrm{Re}\,\lambda : \lambda \in \lambda(\widetilde{M}_{\mathcal{F}})\} < 0.$$

- **Discrete / monodromy classes** (`dt`, `dtmjls`, `sampled`, `impulsive`, `switched`): the solution of $X(k{+}1) = \mathcal{J}(X(k))$ is $\widetilde{M}_{\mathcal{J}}^k\,\mathrm{svec}(X_0)$, so it decays iff the **spectral radius** is below one, equivalently the log spectral radius is negative,

$$\varrho(\mathcal{J}) = \max\{|\lambda| : \lambda \in \lambda(\widetilde{M}_{\mathcal{J}})\} < 1 \iff \log\varrho(\mathcal{J}) < 0.$$

3.1.4 Monodromy operators for dwell classes

A dwell-time or periodic system alternates a jump-free flow of length θ with a reset. The pre-jump samples $\zeta_k = X(t_k^-)$ obey the discrete recursion $\zeta_{k+1} = \Psi_\theta(\zeta_k)$ with the **monodromy operator**

$$\Psi_\theta = \mathcal{U}(\theta, 0) \circ \mathcal{J},$$

where $\mathcal{U}(\theta, 0)$ is the flow over one interval. For constant data the reduced monodromy matrix is the explicit product

$$\widetilde{M}_{\mathcal{J}} e^{\widetilde{M}_{\mathcal{F}}\theta},$$

and stability is decided by its spectral radius. The order of the two factors is immaterial to the verdict: $\widetilde{M}_{\mathcal{J}} e^{\widetilde{M}_{\mathcal{F}}\theta}$ and $e^{\widetilde{M}_{\mathcal{F}}\theta}\widetilde{M}_{\mathcal{J}}$ share the same spectrum (as AB and BA always do), so either product may be formed. This is the matrix `mvst_stability` forms for the dwell-time classes; crossing $\varrho = 1$ (log radius zero) marks the **critical dwell-time**.

3.1.5 Dwell-time notions

The admissible jump sequences are constrained through the spacings $\Delta_k = t_{k+1} - t_k$:

- **Arbitrary**: no constraint (certified by a single constant P).
- **Constant (periodic)**, $\Delta_k \equiv \bar{T}$: one periodic sequence.
- **Minimum**, $\Delta_k \geq \bar{T}$: jumps never closer than $\bar{T}$.
- **Range**, $\Delta_k \in [T_{\min}, T_{\max}]$.

(The chapter also treats maximum $\Delta_k \leq \bar{T}$, average $\kappa(t,s) \leq N_0 + (t-s)/\tau^*$, and persistent dwell-time; the SOS engine covers the first four.)

3.1.6 Margin conventions

All margins the toolbox reports are **second-moment (mean-square) rates**, because the analysis runs on the lifted second-moment operator. For the scalar diffusion $\dot{x} = ax$ the covariance obeys $\dot{X} = 2aX$: the second moment decays at rate $2a$, twice the deterministic state exponent a. So a reported margin is twice the corresponding first-moment exponent; halve it to recover the deterministic rate. A **negative** margin always means mean-square stable:

- continuous-flow classes: margin $= \mu(\mathcal{F})$, the spectral abscissa;
- discrete / monodromy classes: margin $= \log \varrho$, the log spectral radius.

The `res.kind` field names the criterion used.

3.1.7 Timer-dependent certificates

For dwell-time (not just periodic) regimes, stability is certified by a **timer-dependent** Lyapunov matrix $P(\tau)$, $\tau = t - t_k$, satisfying a differential LMI (DLMI). The representative periodic condition is

$$\dot{P}(\tau) + \mathcal{F}^*(\tau, P(\tau)) \preceq -\alpha_3 I \text{ on } (0, T_p), \qquad \mathcal{J}^*(P(0)) - P(T_p) \preceq -\alpha_4 I,$$

which is necessary and sufficient for exponential stability. The DLMI is made finite by seeking $P(\tau) = \sum_{i=0}^{d} P_i \tau^i$ and enforcing the inequalities as **matrix sum-of-squares** conditions in τ, localized to the interval by a multiplier $\tau(\bar{T} - \tau) \geq 0$. This is what `mvst_stab_sos` solves; the `.deg` option sets the polynomial degree d.

For deeper theory (Perron-Frobenius structure, looped functionals, the delay case) see the companion monograph.

3.2 Exact analysis: `S.stability`

`S.stability` decides mean-square (exponential) stability from an exact spectral test on the relevant lifted second-moment operator (the reduced representation on the symmetric subspace). No sum-of-squares and no optimization are used. The system is treated as autonomous: the drift is the flow or closed-loop generator, and any input matrix is ignored.

`res.stable` is the decision, `res.margin` the margin (abscissa or log-radius as above), `res.spectrum` the lifted eigenvalues, and `res.kind` the criterion.

3.2.1 Continuous-time with state-multiplicative noise

The lifted generator here is $\widetilde{M}_{\mathcal{F}}$ of $\dot{X} = \mathrm{He}(AX) + A_c X A_c^*$, and its abscissa is the margin.

```
S = mvst.System.ct([0 1;-2 -1], zeros(2,0), 'Ac', 0.15*eye(2));
r = S.stability();
fprintf('stable=%d  margin=%.6f  kind=%s\n', r.stable, r.margin, r.kind);
```

```
stable=1  margin=-0.977500  kind=ct:spectral-abscissa
```

The margin -0.9775 is the second-moment rate; the deterministic state exponent is about -0.49.

3.2.2 Switched system on a dwell grid

For a switched system the decision is for the **single periodic schedule** that visits the modes in index order (1, 2, ..., M) with the same dwell-time `T` in each mode (the monodromy over one such cycle), not a certificate for arbitrary or worst-case switching. Pass the dwell-time through `OPTS.T`; a vector produces a stability-vs-dwell-time curve in `res.curve`.

```
S = mvst.System.switched({[-1 1;0 -2],[-2 0;1 -1]}, {zeros(2,0),zeros(2,0)});
r = S.stability(struct('T',0.5));
fprintf('stable=%d  margin=%.6f  kind=%s\n', r.stable, r.margin, r.kind);
```

```
stable=1  margin=-2.000000  kind=switched:monodromy-log-radius
```

3.2.3 Impulsive system

The margin is the log spectral radius of the one-cycle monodromy $\widetilde{M}_{\mathcal{J}} e^{\widetilde{M}_{\mathcal{F}} T}$.

```
S = mvst.System.impulsive([0 1;-1 0], zeros(2,0), 0.5*eye(2));
r = S.stability(struct('T',0.5));
fprintf('stable=%d  margin=%.6f  kind=%s\n', r.stable, r.margin, r.kind);
```

```
stable=1  margin=-1.386294  kind=impulsive:monodromy-log-radius
```

A pure discrete-time single-mode system has no dedicated factory; request it with `OPTS.class = 'dt'`.

3.3 Case study: stability versus dwell-time

This end-to-end study builds a system with a flow/jump trade-off, sweeps the dwell-time to trace the stability curve, locates the critical dwell-time where the monodromy radius crosses one, and confirms it against a closed-form prediction.

Take an **unstable** oscillatory flow damped only by a **contractive periodic reset**:

$$A = \begin{bmatrix} 0 & 1 \\ 2 & 0.1 \end{bmatrix} \quad \text{(unstable)}, \qquad J = 0.4 I_2 \quad \text{(contractive)}.$$

Over one dwell-time interval of length T the second-moment monodromy grows through the flow (lifted abscissa $a = \mu(\mathcal{F}) > 0$) and contracts through the reset (lifted radius $\rho = \varrho(\mathcal{J}) < 1$), so its radius is roughly ρe^{aT} and the log radius crosses zero at the critical dwell-time $T^* = \log(1/\rho)/a$: stable for $T < T^*$, unstable for $T > T^*$.

```
A0 = [0 1; 2 0.1];  J = 0.4*eye(2);
S  = mvst.System.impulsive(A0, zeros(2,0), J);

a   = max(real(eig(mvst_liftsym(A0,{}))));       % lifted flow abscissa
rho = max(abs(eig(mvst_liftsym(zeros(2),{J}))));  % lifted reset radius
fprintf('lifted flow abscissa a   = %.6f\n', a);
fprintf('lifted reset radius  rho = %.6f\n', rho);

Tgrid = linspace(0.05, 1.2, 240);
r = S.stability(struct('T', Tgrid));       % stability-vs-dwell-time curve
idx = find(r.curve.margin < 0, 1, 'last');
Tstar = interp1(r.curve.margin(idx:idx+1), Tgrid(idx:idx+1), 0);
fprintf('critical dwell-time  T* (from curve)      = %.6f\n', Tstar);
fprintf('critical dwell-time  T* = log(1/rho)/a    = %.6f\n', log(1/rho)/a);
```

```
lifted flow abscissa a   = 2.930194
lifted reset radius  rho = 0.160000
critical dwell-time  T* (from curve)      = 0.625413
critical dwell-time  T* = log(1/rho)/a    = 0.625413
```

The critical dwell-time obtained from the swept curve matches the closed-form $\log(1/\rho)/a$ to all printed digits. Sampling a few points across T^* shows the sign change of the log radius:

```
for T = [0.2 0.5 0.6244 0.8 1.0]
    rr = S.stability(struct('T', T));
    fprintf('  T=%.4f  logradius=%+.4f  stable=%d\n', T, rr.margin, rr.stable);
end
```

```
  T=0.2000  logradius=-1.2465  stable=1
  T=0.5000  logradius=-0.3675  stable=1
  T=0.6244  logradius=-0.0030  stable=1
  T=0.8000  logradius=+0.5116  stable=0
  T=1.0000  logradius=+1.0976  stable=0
```

Interpretation: frequent resets (small T) keep the second moment contracting despite the unstable flow, but once the dwell-time exceeds $T^* \approx 0.625$ the flow growth over one interval outpaces the fixed per-jump contraction and the mean square diverges. This is the mechanism behind every dwell-time theorem: stability is a race between continuous growth and discrete contraction, decided by the monodromy radius.

The same call structure handles a **sample-and-hold reset** that damps only the velocity, $J = \mathrm{diag}(1, 0.3)$ on a neutrally stable oscillator $A = \begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix}$. Sweeping the sampling period gives a non-monotone margin that is stable across the whole grid, best near $T \approx 0.87$ and degrading at both short and long periods:

```
Ss = mvst.System.impulsive([0 1;-1 0], zeros(2,0), [1 0;0 0.3]);
for T = [0.2 0.5 1.0 2.0 3.0]
    rr = Ss.stability(struct('T', T));
    fprintf('  T=%.3f  logradius=%+.4f  stable=%d\n', T, rr.margin, rr.stable);
end
```

```
  T=0.200  logradius=-0.0768  stable=1
  T=0.500  logradius=-0.6301  stable=1
  T=1.000  logradius=-1.2040  stable=1
  T=2.000  logradius=-1.2040  stable=1
  T=3.000  logradius=-0.0378  stable=1
```

3.4 Dwell-time SOS analysis: `mvst_stab_sos`

The sum-of-squares stability test certifies stability of an impulsive system under a dwell-time regime by a matrix-SOS program in the timer (SOSTOOLS + SeDuMi). It is the computational form of the timer-dependent DLMI certificate above: it searches for a polynomial $P(\tau) = \sum_i P_i \tau^i$ making the flow and jump inequalities hold as SOS matrices on the dwell-time interval. The dwell-time struct selects the regime: `'arbitrary'`, `'discrete'`, `'periodic'` (fixed `Tbar`), `'minimum'` (`Tbar`), or `'range'` (`[Tmin,Tmax]`), and `.deg` sets the timer polynomial degree of the certificate.

The following certifies a fixed periodic dwell-time for a marginally stable flow with a contractive reset:

```
sys = struct('A0',[0 1;-1 0], 'J', 0.5*eye(2));
res = mvst_stab_sos(sys, struct('type','periodic','Tbar',0.5,'deg',4));
fprintf('feas=%d  slack=%.3e\n', res.feas, res.slack);
```

```
feas=1  slack=5.000e-01
```

`res.feas` is the feasibility flag and `res.slack` the certificate slack (a positive value here confirms a strictly feasible storage certificate). The product-cone variant `mvst_stab_sos_modes` handles multi-mode switched systems.

This routine requires SOSTOOLS and an SDP backend (see Getting started). The exact `S.stability` needs only base MATLAB, and the case study above uses only the exact engine.

3.5 Further stability engines

These two engines are called directly as `mvst_*` functions (there is no `mvst.System` method for them).

3.5.1 `mvst_stab_critical`: critical parameter on the stability margin

`res = mvst_stab_critical(sys, param, range, opts)` finds, by bisection, the value of a scalar parameter in `range = [lo hi]` at which the exact mean-square stability margin of `sys` (from `mvst_stability`) crosses zero. By default `param` is forwarded as an `opts` field to `mvst_stability`, so the natural use is the dwell-time or sampling period `'T'`; for an arbitrary parameter (a noise scale, a coupling rate) supply `opts.setter = @(sys,x) sys_rebuilt_at_x`, and `param` is then only a label.

```
sys = struct('A',[0.3 1;0 0.3],'J',0.5*eye(2));   % unstable flow, contracting reset
res = mvst_stab_critical(sys,'T',[0.01 5],struct('class','impulsive'));
fprintf('crit=%.4f margin_at=%.1e\n',res.crit,res.margin_at);
```

```
crit=2.3105 margin_at=2.1e-09
```

The system is stable for dwell-time periods below `res.crit = 2.3105` and unstable above; `res.margin_at` is the margin at the returned value (zero up to the bisection tolerance). If the margin does not change sign on `range`, the function errors rather than returning an end point.

3.5.2 `mvst_stab_delay`: delay-independent stability

`res = mvst_stab_delay(sys, opts)` certifies stability for all delays `h_i >= 0` at once, by a Lyapunov-Krasovskii LMI and, equivalently, the spectrum of the zero-delay operator. `opts.domain` is `'ct'` (default), `'dt'`, or `'positive'`; `sys` carries the drift `A0`, the drift channels, and a cell `Ad` of delayed blocks. It is a SOSTOOLS program by default (any backend, including

'nosdp'); the legacy YALMIP formulation is `opts.engine = 'yalmip'` (optional external dependency).

```
C = struct('A0',diag([-2 -3]),'Ad',{{diag([0.5 0.5])}});
r = mvst_stab_delay(C,struct('domain','ct'));
fprintf('stable=%d margin=%.4f\n',r.stable,r.margin);
```

```
stable=1 margin=-3.7500
```

`res.margin` is the spectral abscissa of the zero-delay operator; it is negative here, so the system is stable for every delay.

Chapter 4

Stabilization

This chapter covers state-feedback stabilization: the exact Riccati routes of `mvst_stabilize` (method `S.stabilize`) and the sum-of-squares synthesis of `mvst_stabz_sos`. It corresponds to the companion monograph chapter **Stabilization**.

It opens with the mathematical setting (the feedback structure, the closed-loop covariance generator, the Riccati equation the exact route solves, and the convexified change of variables behind the dwell-time synthesis), documents both engines, and closes with an end-to-end case study that designs, certifies, and interprets a family of feedbacks up to the mean-square stabilizability limit.

4.1 Mathematical setting

4.1.1 State-feedback structure

The controlled diffusion of Systems,

$$\mathrm{d}x = (Ax + Bu)\,\mathrm{d}t + \sum_{l=1}^{q} (A_l x + B_l u)\,\mathrm{d}W_l,$$

is closed with a **linear state feedback** $u = Kx$. Substituting collapses drift and channels into closed-loop coefficients

$$A_{\mathrm{cl}} = A + BK, \qquad A_{l,\mathrm{cl}} = A_l + B_l K,$$

so the closed-loop second moment $X = \mathbb{E}[xx^*]$ obeys the covariance ODE

$$\dot{X} = \mathrm{He}\left(A_{\mathrm{cl}} X\right) + \sum_{l=1}^{q} A_{l,\mathrm{cl}} X A_{l,\mathrm{cl}}^* = \mathcal{F}_{\mathrm{cl}}(X),$$

with $\mathrm{He}(M) = M + M^*$. **Mean-square stabilization** is the design of K making $\mathcal{F}_{\mathrm{cl}}$ mean-square exponentially stable, that is (from Stability) making the lifted generator $\widetilde{M}_{\mathcal{F}_{\mathrm{cl}}}$ have negative spectral abscissa. Note that the input channels B_l make the noise itself feedback-dependent: a gain that shrinks the drift can amplify the multiplicative noise, so a plant need not be mean-square stabilizable even when its deterministic part is.

4.1.2 The Riccati route

For the classes with an exact route, the optimal stabilizing gain comes from the stabilizing solution $P \succ 0$ of the continuous-time **indefinite / stochastic algebraic Riccati equation** (the I-GRE). This is exactly what the engines `mvst_ric_ct` / `mvst_care_ct` compute on real

data (see the data-scope note in Systems), so it is written here in the real (transpose) form the code uses:

$$\mathrm{He}(A^\top P) + \sum_{l=1}^{q} A_l^\top P A_l + Q - S^\top \Theta^{-1} S = 0,$$

with the noise-weighted quantities

$$\Theta = R + \sum_{l=1}^{q} B_l^\top P B_l, \qquad S = B^\top P + S_\times^\top + \sum_{l=1}^{q} B_l^\top P A_l,$$

where $Q = Q^\top \succeq 0$ and $R = R^\top \succ 0$ are the state and input weights and $S_\times$ the cross weight. Here $\mathrm{He}(A^\top P) = A^\top P + PA$. The stabilizing gain is the completion of squares

$$K = -\Theta^{-1} S, \qquad u = Kx.$$

Two features distinguish it from the deterministic ARE: the diffusion channels add $\sum_l A_l^\top P A_l$ to the state term, and the input channels make the control weight $\Theta = R + \sum_l B_l^\top P B_l$ state-solution-dependent. When $R + \sum_l B_l^\top P B_l$ fails to stay positive definite the design has hit the mean-square stabilizability limit and no stabilizing $P \succ 0$ exists.

The toolbox obtains P by the monotone-limit construction of the monograph: integrate the differential I-GRE $\dot{P} = -\mathcal{R}(P)$ backward from $P(T) = 0$ over a growing horizon until the residual $\|\mathcal{R}(P)\|$ is negligible; P_T increases monotonically to P_∞. This needs only ODE integration.

Per-class variants keep the same shape: the `mjls` class solves the **coupled** Riccati system (one P_i per mode, coupled through $\sum_j \pi_{ji} P_j$ added to the state term), the `poisson` class adds the jump term $\sum_r \lambda_r (J_r^* P J_r - P)$, and the `sampled` class solves the periodic sampled-data I-GRE over one sampling interval.

4.1.3 Convexified dwell-time synthesis

The Riccati route gives a single constant gain. Under a dwell-time constraint the gain is allowed to depend on the timer $\tau = t - t_k$, and the synthesis is made convex by a linearizing change of variables. The closed-loop timer-dependent DLMI in a Lyapunov matrix $P(\tau)$ is bilinear in (P, K), so one works with the **dual** variables

$$Y(\tau) = P(\tau)^{-1} \succ 0, \qquad U(\tau) = K(\tau) Y(\tau),$$

which turns the closed-loop condition into an LMI/SOS in (Y, U): schematically, for the flow part,

$$-\dot{Y}(\tau) + \mathrm{He}\big(AY(\tau) + BU(\tau)\big) + \Xi\big(Y, U\big) \preceq 0,$$

where Ξ collects the diffusion channels via a Schur complement, together with a jump condition linking $Y(0)$ and $Y(\bar{T})$ through the reset. The gain is recovered as $K(\tau) = U(\tau) Y(\tau)^{-1}$. Seeking polynomial $Y(\tau)$, $U(\tau)$ and enforcing the inequalities as matrix sum-of-squares in τ (as in Stability) makes the search finite. This is what `mvst_stabz_sos` solves, returning the timer-dependent gain as a function handle.

For deeper theory (existence, the maximal solution, dwell-time necessity) see the companion monograph.

4.2 Exact synthesis: `S.stabilize`

`S.stabilize` designs a mean-square stabilizing feedback `u = K x` by the closed-form / Riccati route of the resolved class, then **certifies** the closed loop by an exact call to `mvst_stability`. The design is only accepted when `res.check.stable` is true. Routing by class mirrors the book:

- `ct`: a single stochastic algebraic Riccati (`mvst_care_ct`);
- `mjls`: the coupled continuous-time MJLS Riccati (`mvst_solve_infinite`);
- `switched`: per-mode infinite-horizon gains (valid for a large-enough dwell-time);
- `sampled`: the periodic sampled-data impulsive Riccati (`mvst_igre_periodic_sd`);
- `poisson`: the GBM + Poissonian-jump algebraic Riccati (`mvst_care_ct_poisson`).

4.2.1 Continuous-time with multiplicative noise

```
S = mvst.System.ct([0 1;-1 0.2], [0;1], 'Q', eye(2), 'R', 1, ...
                   'Ac', 0.3*[0 0;1 0], 'Bc', [0;0.2]);
r = S.stabilize();
disp(r.K); fprintf('closed-loop stable=%d\n', r.check.stable);
```

```
   -0.5040   -1.5421
closed-loop stable=1
```

`res.K` is the gain (a matrix for single-mode classes, a cell of gains for the multi-mode classes), `res.P` the Riccati solution P, `res.closedloop` the autonomous closed-loop struct fed to the check, and `res.check` the stability result on it (its `.margin` is the certified second-moment abscissa).

4.2.2 Markov-jump system

```
Pi = [-1 1; 2 -2];
S  = mvst.System.mjls({[0 1;-1 0.2],[0 1;-2 -0.1]}, {[0;1],[0;1.2]}, Pi, ...
                      'Q', eye(2), 'R', 1);
r = S.stabilize();
fprintf('closed-loop stable=%d\n', r.check.stable);
```

```
closed-loop stable=1
```

The discrete-time (`dt`, `dtmjls`) and the dwell-time-constrained impulsive designs are handled by the SOS engine below, which `S.stabilize` deliberately does not duplicate.

4.3 Case study: the mean-square stabilizability limit

This end-to-end study designs a feedback for a family of increasingly noisy plants, certifies each closed loop, and locates the noise level beyond which no gain can stabilize the mean square. It illustrates the point made above: state- and input-multiplicative noise scale with the design, so a deterministically controllable plant can lose mean-square stabilizability.

Take the unstable double-integrator-like plant

$$A = \begin{bmatrix} 0 & 1 \\ 1 & 0.5 \end{bmatrix}, \qquad B = \begin{bmatrix} 0 \\ 1 \end{bmatrix},$$

with open-loop eigenvalues $\{1.2808, -0.7808\}$ (deterministically controllable), and attach a common noise level σ to both a state channel and the input channel, $A_c = \sigma \begin{bmatrix} 0 & 0 \\ 1 & 0 \end{bmatrix}$ and

$B_c = \sigma \begin{bmatrix} 0 \\ 1 \end{bmatrix}$. We sweep σ, solve the I-GRE each time, and read the certified closed-loop second-moment margin.

```
Abase = [0 1; 1 0.5];  B = [0;1];
fprintf('open-loop deterministic eig: %s\n', mat2str(round(eig(Abase),4)'));
for sigma = [0.0 0.3 0.6 0.9 1.2 1.5 1.8]
    S = mvst.System.ct(Abase, B, 'Q', eye(2), 'R', 1, ...
                       'Ac', sigma*[0 0;1 0], 'Bc', sigma*[0;1]);
    r = S.stabilize();
    fprintf('  sigma=%.2f  K=[% .3f % .3f]  cl-margin=%+.4f  stable=%d\n', ...
            sigma, r.K(1), r.K(2), r.check.margin, r.check.stable);
end
```

```
open-loop deterministic eig: [-0.7808 1.2808]
  sigma=0.00  K=[-2.414 -2.965]  cl-margin=-1.8162  stable=1
  sigma=0.30  K=[-2.228 -2.729]  cl-margin=-1.5483  stable=1
  sigma=0.60  K=[-1.716 -2.066]  cl-margin=-0.7905  stable=1
  sigma=0.90  K=[-1.126 -1.225]  cl-margin=-0.0985  stable=1
  sigma=1.20  K=[-1.000 -0.694]  cl-margin=+0.3056  stable=0
  sigma=1.50  K=[-1.000 -0.444]  cl-margin=+0.5556  stable=0
  sigma=1.80  K=[-1.000 -0.309]  cl-margin=+0.6914  stable=0
```

Interpretation of the sweep: for $\sigma \leq 0.9$ the I-GRE has a stabilizing solution and the certified second-moment margin is negative (mean-square stable), but the margin degrades toward zero as the noise grows, and the gain magnitude shrinks because pushing harder would only inject more multiplicative noise. Between $\sigma = 0.9$ (margin -0.0985) and $\sigma = 1.2$ (margin $+0.3056$) the closed loop crosses into mean-square instability: the control weight $R + \sum_l B_l^\top P B_l$ can no longer be balanced against the state term enlarged by the noise, the Riccati residual blows up (the solver warns that the residual exceeds tolerance), and the returned gain no longer certifies. The plant is **deterministically controllable at every** σ yet loses **mean-square stabilizability** past $\sigma \approx 1$. This threshold is invisible to a deterministic design and is exactly what the matrix-valued (second-moment) formulation captures.

4.4 SOS synthesis: `mvst_stabz_sos`

`mvst_stabz_sos` searches (SOSTOOLS + SeDuMi) for a stabilizing gain by the bounded-real change of variables of the mathematical setting, covering every dwell-time type of the chapter (`'arbitrary'`, `'discrete'`, `'periodic'`, `'minimum'`, `'range'`). It returns `res.feas`, `res.slack`, `res.Kfun` (a function handle `@(tau) -> K(tau)`, the recovered $U(\tau)Y(\tau)^{-1}$), and `res.Kf` (the spontaneous-jump gain).

The example below stabilizes an unstable continuous-time drift with a state-multiplicative Brownian channel under the arbitrary-dwell-time (constant certificate) program, and certifies the design with the exact lifted-operator abscissa:

```
sysz = struct('A0',[0 1; 2 0.1], 'B0',[0;1], 'Ac', 0.3*eye(2));
rz = mvst_stabz_sos(sysz, struct('type','arbitrary'));
K  = rz.Kfun(0);
absOpen   = max(real(eig(mvst_liftsym([0 1;2 0.1], {0.3*eye(2)}))));
absClosed = max(real(eig(mvst_liftsym([0 1;2 0.1]+[0;1]*K, {0.3*eye(2)}))));
fprintf('feas=%d  K=[% .4f % .4f]\n', rz.feas, K(1), K(2));
fprintf('open-loop abscissa=%.4f  closed-loop abscissa=%.4f\n', absOpen, absClosed);
```

```
feas=1  K=[-3.5002 -1.4417]
open-loop abscissa=3.0202  closed-loop abscissa=-1.2517
```

The open-loop lifted abscissa is positive (mean-square unstable) and the closed-loop one is negative (mean-square stable), so the synthesized gain is certified. `mvst_liftsym` builds the reduced second-moment operator used for the check.

This routine requires SOSTOOLS and an SDP backend; `S.stabilize` and the case study above need only base MATLAB.

4.5 Aperiodic sampled-data refresh gains

4.5.1 `mvst_stabz_sd_range`: range sampling

`res = mvst_stabz_sd_range(plant, opts)` designs the refresh gain of a held-input sampled-data plant whose sampling period is only known to lie in a window `[Tmin, Tmax]`, as a matrix sum-of-squares program in the timer. `opts.mode` is `'scheduled'` (a period-dependent gain `K_J(theta)`), `'constant'` (a schedule-free gain), or `'slack'` (a constant gain with a period-dependent certificate, less conservative than `'constant'`); `opts.deg` is the certificate degree. The plant carries `A`, `Bu`, and optionally the Brownian channels `Ac`, `Bc`. It requires SOSTOOLS and an SDP backend.

```
plant = struct('A',[0 1;2 0.1],'Bu',[0;1],'Ac',0.2*eye(2),'Bc',0.1*[0;1]);
r = mvst_stabz_sd_range(plant,struct('Tmin',0.2,'Tmax',0.3,'mode','scheduled','deg'
    ↪ ,4));
fprintf('feas=%d slack=%.4g\n',r.feas,r.slack);
```

```
feas=1 slack=0.005661
```

`res.feas` and `res.slack` are the feasibility flag and certificate slack, `res.KJfun` the gain as a function of the period, and `res.KJ` a representative value of it (see `help mvst_stabz_sd_range` for the exact convention).

Chapter 5

Optimal control

This chapter covers linear-cost optimal control over the three horizons and the sum-of-squares (SOS) layer that reproduces it by a dual storage-function program. It corresponds to the book chapter **Linear-Cost Optimal Control** of the companion monograph. The section below states the problem in full so that the chapter is self-contained; no access to the book is assumed.

5.1 Mathematical setting

The system. The object of study is a linear Hermitian matrix-valued system. The concrete case is a controlled diffusion whose state `x(t)` in `C^n` and control `u(t)` in `C^m` evolve as

$$\mathrm{d}x = (Ax + Bu)\,\mathrm{d}t + \sum_l (A_l x + B_l u)\,\mathrm{d}W_l,$$

with independent scalar Brownian increments `dW_l`; spontaneous (Poisson) jumps `x^+ = J_r x + B_{J,r} u` firing at rate `lambda_r` may be added. The toolbox carries `A`, `B` (drift and control input), the multiplicative-noise channels `A_l`, `B_l` (`Ac`, `Bc`), and the jump data (`Jf`, `lamf`, `BJf`).

Why matrices. All second-order information is carried by the joint second-moment matrix

$$X(t) = \mathbb{E}\begin{bmatrix} x \\ u \end{bmatrix}\begin{bmatrix} x \\ u \end{bmatrix}^* = \begin{bmatrix} X_1 & X_2 \\ X_2^* & X_3 \end{bmatrix} \succeq 0, \qquad X_1 = \mathbb{E}[xx^*], X_3 = \mathbb{E}[uu^*].$$

`X(t)` is a Hermitian positive-semidefinite matrix trajectory, and every cost and constraint of the theory is **linear** in `X`.

The linear cost. With the augmented cost weight

$$Z = \begin{bmatrix} Q & S \\ S^* & R \end{bmatrix} \succeq 0, \qquad Q = Q^* \succeq 0, R = R^* \succ 0,$$

the running cost is the inner product `<Z,X> = tr(ZX)`, which unfolds to the familiar LQ integrand

$$\langle Z, X\rangle = \mathrm{tr}(QX_1) + 2\Re\,\mathrm{tr}(S^*X_2) + \mathrm{tr}(RX_3) = \mathbb{E}\left[x^*Qx + 2\Re(x^*Su) + u^*Ru\right].$$

The **finite-horizon** cost over `[0,T]`, with terminal weight `Z_T` on the state block `X_1(T)`, is

$$J_T = \int_0^T \langle Z, X(t)\rangle\,\mathrm{d}t + \langle Z_T, X_1(T)\rangle.$$

The **infinite-horizon** cost and the **long-term average** cost rate are

$$J_\infty = \int_0^\infty \langle Z, X(t)\rangle\,\mathrm{d}t, \qquad \rho = \lim_{T\to\infty}\frac{1}{T}J_T = \sum_i \pi_i^{\mathrm{ss}}\langle P_i, W_i\rangle,$$

where `W_i` is the additive process-noise intensity of mode `i` and `pi^ss` is the stationary distribution of the switching Markov chain (for a single mode the sum is one term <P,W>). The average-cost problem is the right one under persistent excitation: `J_infty` is then infinite, but `rho` is finite.

The impulsive generalized Riccati equation (I-GRE). Optimality is certified by a dual variable `P = P^*` in `C^{n x n}`, using the Hermitian-part operator `He(M) = M + M^*`, through the three flow blocks

$$(\mathcal{C}_Z)_1 = \mathrm{He}(A^*P) + Q + \textstyle\sum_l A_l^* P A_l + \sum_r \lambda_r (J_r^* P J_r - P),$$
$$(\mathcal{C}_Z)_2 = PB + S + \textstyle\sum_l A_l^* P B_l + \sum_r \lambda_r J_r^* P B_{J,r},$$
$$(\mathcal{C}_Z)_3 = R + \textstyle\sum_l B_l^* P B_l + \sum_r \lambda_r B_{J,r}^* P B_{J,r},$$

In the nonsingular case `(C_Z)_3 > 0` the finite- and infinite-horizon I-GRE are the backward differential and the algebraic Riccati equations

$$-\dot{P} = (\mathcal{C}_Z)_1 - (\mathcal{C}_Z)_2 (\mathcal{C}_Z)_3^{-1} (\mathcal{C}_Z)_2^*, \quad P(T) = Z_T \qquad \text{and} \qquad (\mathcal{C}_Z)_1 - (\mathcal{C}_Z)_2 (\mathcal{C}_Z)_3^{-1} (\mathcal{C}_Z)_2^* = 0.$$

The optimal state-feedback gain and the optimal cost are

$$K^\star = -(\mathcal{C}_Z)_3^{-1} (\mathcal{C}_Z)_2^*, \qquad J^\star = \langle P, X_0 \rangle = \mathrm{tr}(P X_0),$$

with `X_0 = X_1(0)` the initial second moment. The singular case (`(C_Z)_3` rank-deficient, free or singular inputs) is handled by the pseudoinverse form; the book develops it, and the SOS layer certifies it without inverting anything. **Standing assumptions:** `(A,B)` stabilizable and `(A, Q^{1/2})` detectable for the infinite-horizon problem, `Q >= 0`, `R > 0`.

What the solver returns. Each engine returns the dual `P` (the I-GRE solution), the gain `K^star`, and the cost: `.cost = <P,X0>` for the finite and infinite horizons, `.rho` for the average horizon.

5.2 The three horizons

A system carries the quadratic-cost blocks `Q`, `R` (and optionally `S`, the terminal weight `ZT`, the initial covariance `X0`, and the noise intensity `W`). Three methods price the three horizons of the chapter:

- `S.ocfinite(T)`: the finite-horizon differential I-GRE on `[0,T]` integrated backward from `P(T) = Z_T`, via `mvst_solve_finite`;
- `S.ocinfinite()`: the infinite-horizon (algebraic, coupled) I-GRE, via `mvst_solve_infinite`;
- `S.ocaverage()`: the long-term average cost rate `rho`, via `mvst_solve_average`.

5.2.1 Finite and infinite horizon

The finite-horizon cost is `<P(0), X0>` for the built initial covariance; here `X0 = I`, so the reported cost is `trace(P(0))`. As the horizon grows it approaches the infinite-horizon cost:

```
S = mvst.System.ct([-1 0;0 -2], [1;1], 'Q', eye(2), 'R', 1, 'X0', eye(2));
fprintf('finite cost(T=3) = %.6f\n', S.ocfinite(3).cost);
fprintf('infinite cost = %.6f\n', S.ocinfinite().cost);
```

```
finite cost(T=3) = 0.660876
infinite cost = 0.661064
```

The finite-horizon value (0.660876) sits just below the infinite-horizon value (0.661064), as expected for a converging horizon: the finite horizon accrues strictly less cost than the infinite one, and closes the gap as `T` grows.

5.2.2 Average cost

The long-term average cost rate `rho` weights the stationary running cost by the noise intensity `W`:

```
S = mvst.System.ct([-1 0;0 -2], [1;1], 'Q', eye(2), 'R', 1, 'W', eye(2));
fprintf('average rho = %.6f\n', S.ocaverage().rho);
```

```
average rho = 0.661064
```

`S.ocaverage` returns the infinite-horizon solution plus the extra field `rho`. Result structs additionally carry the optimal gain `K` and the Riccati solution `P` (both `1 x M` cells, one entry per mode).

5.3 Worked case study: horizon convergence and the closed loop

This end-to-end example prices a single non-normal mode over a sweep of horizons, confirms the monotone convergence to the infinite-horizon value, extracts the optimal gain, verifies that it stabilizes the closed loop, and cross-checks the whole solution against the exact algebraic Riccati engine `mvst_care_ct`. Every number below is produced by the run.

```
S = mvst.System.ct([-1 0.2; 0 -2], [1;0.5], ...
                   'Q', eye(2), 'R', 2, 'X0', eye(2), 'W', 0.3*eye(2));
for T = [0.5 1 2 5 10]
    fprintf('T=%5.1f  cost=%.6f\n', T, S.ocfinite(T).cost);
end
ri = S.ocinfinite();
fprintf('infinite cost = %.6f  (iters=%d, residual=%.2e)\n', ...
        ri.cost, ri.iters, ri.residual);
fprintf('average rho    = %.6f\n', S.ocaverage().rho);
K = ri.K{1};
fprintf('optimal gain K = %s\n', mat2str(K,5));
Acl = [-1 0.2;0 -2] + [1;0.5]*K;
fprintf('eig(A)   = %s\n', mat2str(sort(eig([-1 0.2;0 -2])),5));
fprintf('eig(Acl) = %s\n', mat2str(sort(eig(Acl)),5));
[Pr,Kr] = mvst_care_ct(mvst_mode([-1 0.2;0 -2],[1;0.5],eye(2),2,'X0',eye(2)));
fprintf('||K-Kcare|| = %.1e   ||P-Pcare|| = %.1e\n', ...
        norm(K-Kr,'fro'), norm(ri.P{1}-Pr,'fro'));
```

```
T=  0.5  cost=0.523734
T=  1.0  cost=0.650952
T=  2.0  cost=0.693441
T=  5.0  cost=0.697055
T= 10.0  cost=0.697057
infinite cost = 0.697057  (iters=2, residual=1.53e-11)
average rho    = 0.209117
optimal gain K = [-0.2286 -0.071784]
eig(A)   = [-2;-1]
eig(Acl) = [-2.0173;-1.2472]
||K-Kcare|| = 0.0e+00   ||P-Pcare|| = 0.0e+00
```

Interpretation of the run: the finite-horizon cost climbs monotonically from 0.523734 at `T = 0.5` and settles at 0.697057 by `T = 10`, matching the infinite-horizon value to six digits (the algebraic solve itself needs only two Gauss-Seidel iterations to a residual of 1.5e-11). The average-cost rate `rho = 0.209117` is the noise-weighted stationary running cost, a different quantity from the transient cost. The optimal gain shifts both eigenvalues left (from `-1, -2` to `-1.25, -2.02`), so the closed loop is stable. Finally the Riccati solution `P` and the gain

K recovered from the infinite-horizon engine agree with the exact algebraic Riccati solver to machine precision, so the horizon result and the closed-form result are the same object.

5.4 SOS layer: mvst_oc_sos

mvst_oc_sos solves the same infinite-horizon problem by the dual (storage-function) SOS / LMI program of the chapter. Instead of integrating the Riccati equation it maximizes <P,X0> over all P for which the augmented KYP matrix

$$\mathcal{C}_Z(P) = \begin{bmatrix} (\mathcal{C}_Z)_1 & (\mathcal{C}_Z)_2 \\ (\mathcal{C}_Z)_2^* & (\mathcal{C}_Z)_3 \end{bmatrix} \succeq 0$$

is a matrix sum of squares in the indeterminates [x;u]. This is the pre-Schur (bounded-real) form of the I-GRE, and the maximizer is exactly the stabilizing solution P^star, from which the gain is recovered as K^star = -(C_Z)_3^{-1}(C_Z)_2^*. The routine returns a primal cost, a dual (moment) cost, and the recovered P and K. The multiplicative-noise and Poisson terms enter P linearly, so the program stays affine. The example below builds a thin mode, solves the exact algebraic Riccati with mvst_care_ct, then the SOS dual, and compares:

```
md = mvst_mode([-1 0;0 -2], [1;1], eye(2), 1, 'X0', eye(2));
[Pr,Kr] = mvst_care_ct(md);
r1 = mvst_oc_sos(md);
fprintf('SOS cost primal=%.6f  dual=%.6f\n', r1.cost.primal, r1.cost.dual);
fprintf('||P_sos-P_ric||=%.2e  ||K_sos-K_ric||=%.2e\n', ...
        norm(r1.P-Pr,'fro'), norm(r1.K-Kr,'fro'));
```

```
SOS cost primal=0.661064  dual=0.661064
||P_sos-P_ric||=4.79e-10  ||K_sos-K_ric||=5.32e-10
```

The SOS primal and dual costs coincide (0.661064) and match the infinite-horizon value above, and the recovered P and K agree with the Riccati solution to about 1e-10. The gap between the two is the SOS relaxation gap at this certificate degree, negligible here. The multi-mode / dwell-time variant is mvst_oc_sos_modes, which handles the periodic, minimum, and range dwell-time regimes by a timer-dependent certificate. The SOS layer requires SOSTOOLS and an SDP backend; the Riccati horizons need only base MATLAB.

5.5 Dwell-time optimal control

5.5.1 mvst_oc_dwell: causal minimum and range dwell-time control

res = mvst_oc_dwell(md, jump, opts) computes the causal optimal control of a single-cone linear impulsive system whose inter-jump times obey a minimum or a range dwell-time constraint. md is a mode struct from mvst_mode (flow data A,B,Q,R), jump carries the reset J, the jump input Bd, and the jump cost Qd, Rd; opts.dwell is 'min' or 'range' with opts.Tmin (and opts.Tmax), and opts.X10 the initial second moment. The certificate is the fixed point of the backward flow integration of the differential Riccati equation composed with the jump Riccati map.

```
md   = mvst_mode([-1 0.5;0 -0.8],[0;1],eye(2),1);
jump = struct('J',1.2*eye(2),'Bd',[0;1],'Qd',0.05*eye(2),'Rd',1);
r = mvst_oc_dwell(md,jump,struct('dwell','min','Tmin',1,'X10',eye(2)));
fprintf('Jbound=%.4f feas=%d Kd=%s\n',r.Jbound,r.feas,mat2str(r.Kd,4));
```

```
Jbound=1.0655 feas=1 Kd=[-0.0974 -0.4166]
```

`res.Jbound` is the certified cost bound realized by the recovered policy for every admissible dwell-time sequence, `res.P0` the certificate at the start of a dwell-time interval, `res.Kc` and `res.Kd` the flow and jump gains. As `Tmin` grows, `res.P0` converges to the infinite-horizon solution of `mvst_care_ct`.

Chapter 6

Steering

This chapter covers finite-time optimal covariance steering: driving the state second-moment covariance to a prescribed target over a horizon. It corresponds to the book chapter **Steering** of the companion monograph. The section below states the problem in full so the chapter is self-contained.

6.1 Mathematical setting

The problem. Take the controlled diffusion of the previous chapter,

$$\mathrm{d}x = (Ax + Bu)\,\mathrm{d}t + \sum_l (A_l x + B_l u)\,\mathrm{d}W_l, \qquad x(0) \sim (\cdot, \Sigma_0),$$

and track only its state covariance

$$\Sigma(t) = \mathbb{E}\big[x(t)x(t)^*\big] \in \mathbb{H}^n_{\succeq 0}.$$

Covariance steering asks for a control policy that moves this Hermitian positive-semidefinite matrix from a given start to a given end over a fixed horizon:

$$\text{find } u(\cdot) \text{ such that } \Sigma(0) = \Sigma_0 \longrightarrow \Sigma(T) = \Sigma_f,$$

with `Sigma_f = Sigma_f^* >= 0` the prescribed terminal covariance. Under a linear state feedback `u = K(t) x` the closed-loop covariance obeys the matrix Lyapunov (Riccati-type) flow

$$\dot{\Sigma} = (A + BK)\Sigma + \Sigma(A + BK)^* + \sum_l (A_l + B_l K)\Sigma(A_l + B_l K)^*, \qquad \Sigma(0) = \Sigma_0.$$

Reachability. The classical controllability Gramian below is only the noiseless, vector-state **intuition**: the true reachability object here is operator-valued (the reachable set of the matrix covariance flow), not a single Gramian matrix. As a special-case guide, for the noiseless controllable model the reachable set is governed by the controllability Gramian

$$W_c(T) = \int_0^T e^{At} B B^* e^{A^* t}\,\mathrm{d}t \succ 0.$$

When `(A,B)` is controllable, `W_c(T)` $\succ$ `0` for every `T` $>$ `0` and a large family of targets `Sigma_f` is reachable; targets that are "too small" relative to the process noise, or that ask for contraction faster than the input authority allows, sit outside the feasible set and no exact steering law exists. Because the underlying object is operator-valued, reachability is in practice confirmed by **convergence of the solver** rather than by forming `W_c(T)` explicitly.

The feedback construction. Steering is solved as a two-point boundary-value problem. It minimizes the control energy $\int_0^T u^* R u \, \mathrm{d}t$ subject to the boundary covariances, so `R = R'* > 0` is the **control weight** of that underlying minimum-energy cost. A symmetric terminal multiplier `M` seeds the **backward** generalized Riccati equation

$$-\dot{P} = \mathrm{He}(A^*P) + \sum_l A_l^* P A_l - \big(PB + \textstyle\sum_l A_l^* P B_l\big)\Theta^{-1}\big(PB + \textstyle\sum_l A_l^* P B_l\big)^*, \qquad P(T) = M,$$

with the **noise-weighted** control term $\Theta = R + \sum_l B_l^* P B_l$: the engine `mvst_ric_ct` inverts exactly this Θ, not the bare `R`, so that the input-multiplicative channels raise the effective control weight. The equation produces the time-varying gain

$$K(t) = -\Theta^{-1}\big(B^*P(t) + \textstyle\sum_l B_l^* P(t) A_l\big), \qquad \Theta = R + \textstyle\sum_l B_l^* P(t) B_l.$$

The state term is displayed here in its **minimum-energy specialization** (no state penalty); the engine reads the mode's cost `md.Q` and adds it to the state block when present.

The gain is then propagated **forward** through the closed-loop Lyapunov flow above to obtain `Sigma(T; M)`. The steering law is the multiplier `M` at which the terminal covariance hits the target:

$$\Phi(M) := \Sigma(T; M) - \Sigma_f = 0.$$

This root-finding map `M -> Sigma(T; M)` is the forward-backward (FB) structure; its Jacobian is dense and the map is nonlinear, so the toolbox solves `Phi(M) = 0` by a **damped Newton iteration**. **Standing assumptions:** `(A,B)` controllable (or stabilizable with `Sigma_f` in the reachable set), `Sigma_0` and `Sigma_f` Hermitian positive semidefinite, `R > 0`.

6.2 S.steer

`S.steer(Xf, T)` steers the state covariance from its initial value to the target `Xf` over `[0,T]` by exactly this forward-backward algorithm: for a symmetric terminal multiplier `M` it runs the backward I-GRE from `P(T) = M` to obtain the time-varying gains, propagates the closed-loop covariance forward, and solves `Sigma(T; M) = Xf` by the damped Newton core `mvst_steer_newton`.

The initial covariance defaults to the mode's `X0`. The example steers a two-state diffusion from `Sigma(0) = I` to `Sigma(T) = 0.5 I` over one time unit, with a full-rank input so the target is reachable:

```
S = mvst.System.ct([-1 0;0 -1], eye(2), 'X0', eye(2));
r = S.steer(0.5*eye(2), 1);
fprintf('iters=%d  final gap=%.3e  class=%s\n', r.iters, r.res, r.class);
fprintf('Sigma(0) =\n'); disp(r.Sigma(:,:,1));
fprintf('Sigma(T) =\n'); disp(r.Sigma(:,:,end));
```

```
iters=8  final gap=2.218e-08  class=ct
Sigma(0) =
     1     0
     0     1
Sigma(T) =
    0.5000         0
         0    0.5000
```

Newton converges in 8 iterations to a terminal gap of about 2e-8, and the final covariance is the requested `0.5 I`.

Result fields: `res.M` (optimal terminal multiplier), `res.t` and `res.Sigma` (the covariance trajectory, an `n x n x nt` array), `res.K` (the optimal gain object, shape depending on class), `res.res` (final gap norm), `res.iters`, and `res.hist` (gap-norm history).

6.3 Worked case study: an anisotropic target and the covariance trajectory

The isotropic example above hides the interesting behaviour, because the two axes move together. This case study steers a non-normal, weakly damped diffusion to an **anisotropic, correlated** target and inspects the whole covariance path. The target has off-diagonal correlation and eigenvalues `0.19` and `0.51`, so the two principal axes must contract by different amounts.

The default Newton seed `M0 = 0` is isotropic and can stall on a hard, anisotropic target (the trial step drives the backward Riccati into finite escape and the line search stalls); a seed in the solution basin restores convergence, so `OPTS.M0` is set here.

```
S  = mvst.System.ct([-0.5 1; 0 -0.5], eye(2), 'X0', eye(2));
Xf = [0.4 0.15; 0.15 0.3];
r  = S.steer(Xf, 2, struct('M0', -0.5*eye(2)));
fprintf('iters=%d  final gap=%.3e  class=%s\n', r.iters, r.res, r.class);
fprintf('Sigma(0) eig = %s\n', mat2str(sort(eig(r.Sigma(:,:,1)))',4));
fprintf('Sigma(T) eig = %s\n', mat2str(sort(eig(r.Sigma(:,:,end)))',4));
fprintf('Sigma(T) =\n'); disp(r.Sigma(:,:,end));
fprintf('||Sigma(T) - Xf|| = %.3e\n', norm(r.Sigma(:,:,end)-Xf,'fro'));
```

```
iters=13  final gap=9.182e-08  class=ct
Sigma(0) eig = [1 1]
Sigma(T) eig = [0.1919 0.5081]
Sigma(T) =
    0.4000    0.1500
    0.1500    0.3000
||Sigma(T) - Xf|| = 1.179e-07
```

Interpretation of the run: the covariance starts isotropic (both eigenvalues `1`), and by `t = T` it has been shaped into the requested correlated target, with the two principal axes now at `0.19` and `0.51`. The terminal covariance reproduces `Xf` to a Frobenius error of 1.2e-7, and the damped Newton needs 13 iterations (more than the isotropic case, as expected for the harder off-diagonal target). The full trajectory `r.Sigma` is available for plotting; the companion helpers below draw it.

6.4 Other classes and hard targets

`S.steer` dispatches on class: `ct`, `mjls`, `switched`, `sd` (sampled-data), and `poisson` are supported, with time-varying variants detected automatically when a drift is a function handle. Pass options through the fourth argument:

- `OPTS.X0`: initial covariance (required for `switched`, `sd`, `poisson`);
- `OPTS.M0`: initial Newton multiplier. As the worked example shows, for hard targets the default `M0 = 0` can leave the damped Newton stranded at a large terminal gap; an `M0` in the solution basin restores quadratic convergence. The sampled-data and Poissonian-jump steering examples set it for this reason;
- `OPTS.tol` (default 1e-7) and `OPTS.maxit` (default 60).

The companion figure helpers `mvst_steer_ellipse` and `mvst_steer_components` draw the book's steering figures from a `res` struct: the first plots the covariance confidence ellipses at

start, middle, and end, the second plots the individual covariance entries against time. Steering uses only base MATLAB (no SDP backend).

Chapter 7

Dissipativity

This chapter covers the L2-gain and passivity, and the general dissipativity SOS program. It corresponds to the book chapter **Dissipativity** of the companion monograph. The section below states the framework in full so the chapter is self-contained.

7.1 Mathematical setting

The system with a port. Add a disturbance input `w` and a performance output `z` to the matrix-valued system:

$$\mathrm{d}x = (Ax + B_w w)\,\mathrm{d}t + \sum_l (A_l x + D_{w,l} w)\,\mathrm{d}W_l, \qquad z = Cx + D_w w.$$

The toolbox carries `A` (`A0`), the disturbance input `Bw`, the performance output `C`, the feedthrough `Dw`, and, when present, the multiplicative and jump channels. All second-order information is again carried by the joint second-moment matrix `X = X^* >= 0`, and every quantity below is **linear** in `X`.

The supply rate. Dissipativity meters the power crossing the `(w,z)` port through a quadratic **supply rate**, written compactly as a linear functional of `X`:

$$\sigma(w,z) = \langle M, X\rangle, \qquad M = M^* = \begin{bmatrix} Q_s & S_s \\ S_s^* & R_s \end{bmatrix} \text{ (indefinite, in the state/disturbance partition).}$$

The decisive modelling choice is that `M` carries **no sign**: the right indefinite signature recovers each classical supply. In vector terms `sigma(w,z) = [x;w]^* M [x;w]`, and `M` is built from the port weights and the output map. Throughout, $\mathrm{He}(N) = N + N^*$ (the doubled Hermitian-part operator of Systems) and $\widetilde{C} = \begin{bmatrix} C & D_w \end{bmatrix}$ is the **stacked output map**, so that $z = \widetilde{C}[x; w]$.

The storage and the dissipation inequality. A **linear storage function** is `V(X) = <-P, X>` for a Hermitian `P` (no sign imposed; the sign convention `-P` makes the available storage equal to `<-P(0), X_0>`). The system is **dissipative** with respect to `M` when stored energy never grows faster than the supply delivers, i.e. the dissipation inequality

$$V\big(X(t_1)\big) - V\big(X(t_0)\big) \le \int_{t_0}^{t_1} \sigma\,\mathrm{d}t$$

holds along every admissible trajectory. Differentiating gives the pointwise form `d/dt <-P,X> <= <M,X>`.

The mean-square hybrid `L2 x ell2` gain. For the disturbance-to-output map, the performance measure is the **(mean-square) hybrid `L2 x ell2` gain**: the smallest `gamma >= 0` such that

$$\|z\|_{L_2\times\ell_2} \le \gamma\|w\|_{L_2\times\ell_2}$$

along every trajectory started from rest (`X_1(0) = 0`), where the hybrid norm splits into a flow (`L2`) energy and a jump (`ell2`) energy,

$$\|z\|^2_{L_2\times\ell_2} = \mathbb{E}\left[\int_0^\infty z_c^* z_c \,\mathrm{d}t + \sum_{k\ge 1} z_{d,k}^* z_{d,k}\right].$$

For a purely continuous-time system the jump sum is empty and this is the usual mean-square `L2` gain, the state-space face of the `H_infinity` norm.

The two supplies. Two indefinite signatures recur:

- **`L2`-gain (bounded real):** `sigma = gamma^2 |w|^2 - |z|^2`, whose supply matrix in the state/disturbance partition is

$$M = \begin{bmatrix} 0 & 0 \\ 0 & \gamma^2 I \end{bmatrix} - \widetilde{C}^*\widetilde{C} = \begin{bmatrix} -C^*C & -C^*D_w \\ -D_w^*C & \gamma^2 I - D_w^*D_w \end{bmatrix}.$$

  The system is dissipative with this `M` exactly when its `L2`-gain is at most `gamma`.

- **Passivity:** `sigma = 2\Re\langle z, w\rangle`, i.e.

$$M = \begin{bmatrix} 0 & C^* \\ C & \mathrm{He}(D_w) \end{bmatrix}.$$

Strict refinements subtract an **input-strict** index `nu I` from the `w`-block and an **output-strict** index `rho` times the output quadratic.

The KYP certificate. Dissipativity is certified by a storage matrix `P` (equivalently `S = -P >= 0`) for which the augmented flow operator is positive semidefinite:

$$\mathcal{C}_M(P) = \begin{bmatrix} \mathrm{He}(A^*P) + \sum_l A_l^* P A_l + Q_s & PB_w + \sum_l A_l^* P D_{w,l} + S_s \\ * & \sum_l D_{w,l}^* P D_{w,l} + R_s \end{bmatrix} \succeq 0,$$

where the blocks of `M` are **added** according to the chosen supply, matching $\mathcal{C}_M = \mathcal{F}^*(P) + M$ of the book and the `flow_cM` assembly in `mvst_diss_sos`. As a scalar sanity check, take the `L2`-gain supply with no feedthrough (`Dw = 0`) and no multiplicative channel: the `(2,2)` block reduces to $R_s = \gamma^2$, which is positive, so the certificate can be feasible. Writing a minus there would force $-\gamma^2 < 0$ and make the LMI infeasible for every `gamma`. For the `L2`-gain supply this is the matrix bounded-real lemma; for passivity it is the matrix positive-real (KYP) lemma. **Standing assumptions:** `A` Hurwitz (mean-square stable) for the gain to be finite; `Q >= 0`, `R > 0` where the supply requires them.

7.2 Supply-rate convention in the toolbox

`mvst_diss_sos` selects between the two supplies with `supply.type = 'l2gain' | 'passivity'`. The `'l2gain'` supply builds the `M` above with `gamma` either fixed (`mode = 'feas'`) or minimized (`mode = 'gamma'`); the `'passivity'` supply builds the port-power `M`, with the strict indices `nu` and `rho` available as fields.

7.3 Exact L2-gain: S.l2gain

S.l2gain computes the (mean-square) L2-gain by routing to the exact per-class backend, integrating the bounded-real Riccati rather than relaxing it. Supply the disturbance input Bw and output C through the factory:

```
S = mvst.System.ct([-1 0.5; 0 -2], zeros(2,0), 'Bw', [1;0], 'C', [1 0]);
fprintf('L2 gain = %.6f  method=%s\n', S.l2gain().gamma, S.l2gain().method);
```

```
L2 gain = 1.000001  method=ct
```

res.gamma is the gain and res.method the resolved class. The class is inferred from the data (Poisson jump data, sampling period, generator, or dwell-time), or forced with OPTS.class. The backends are mvst_l2gain_ct, _ct_mjls, _ct_dwell, _ct_poisson, and _sd_hybrid.

7.4 Passivity by frequency sweep: mvst_passivity_ct

For a square continuous-time LTI system, mvst_passivity_ct tests passivity by a frequency sweep of the transfer matrix and returns a passivity index nu. It forms G(jw) = C(jw I - A)^{-1}Bw + Dw, takes the smallest eigenvalue of the Hermitian part over the sweep, and reports nu = min_w lambda_min(G+G^*), the book's convention (supply He(w'z) - nu|w|^2, shared with mvst_passivity and mvst_isp_margin_ct). A positive nu certifies input-strict passivity with that index:

```
[passive,nu] = mvst_passivity_ct([-1 0;0 -2], [1;1], [1 1], 0.5);
fprintf('passive=%d  nu=%.6f\n', passive, nu);
```

```
passive=1  nu=1.000000
```

The signature is positional, mvst_passivity_ct(A, Bw, C, Dw), and the system must be square (size(C,1) == size(Bw,2)) with A Hurwitz. passive flags (approximate) passivity; a negative nu quantifies the shortage of passivity. The sweep is approximate; mvst_passivity (below) gives the exact Hamiltonian test and is the recommended entry point.

7.5 Dissipativity SOS: mvst_diss_sos

mvst_diss_sos(sys, supply, dwell, mode) solves the dual dissipativity DLMIs as a matrix-SOS program in the timer, covering the L2-gain and passivity supplies and every dwell-time type of the chapter. The storage certificate is the dual variable of the chapter (storage matrix S = -P >= 0); the routine poses C_M(P) >= 0 on flow (and its jump counterpart at resets) as a matrix sum of squares. mode is 'gamma' (the supply level is the free variable, minimized) or 'feas' (only certify feasibility at a fixed level).

7.5.1 L2-gain supply, compared to the exact gain

Under the arbitrary-dwell-time (constant certificate, CT LTI) regime the SOS dual reproduces the exact bounded-real gain:

```
A = [-3 0;0 -3]; Bw=[0;1]; C=[0.5 1]; Dw=0.5;
r = mvst_diss_sos(struct('A0',A,'Bw',Bw,'C',C,'Dw',Dw), ...
                  struct('type','l2gain'), struct('type','arbitrary'), 'gamma');
gR = mvst_l2gain_ct_mjls({A},Bw,{C},Dw,0);
fprintf('SOS-dual gamma=%.6f  exact Riccati gamma=%.6f  |diff|=%.2e\n', ...
        r.gamma, gR, abs(r.gamma-gR));
```

```
SOS-dual gamma=0.833334  exact Riccati gamma=0.833986  |diff|=6.52e-04
```

The SOS dual (0.833334) matches the exact Riccati gain (0.833986) to about 6e-4, the residual being the SOS relaxation gap at this certificate degree.

7.5.2 Passivity supply

```
rp = mvst_diss_sos(struct('A0',[-2 1;0 -3],'Bw',[1;0],'C',[1 0],'Dw',1), ...
                   struct('type','passivity'), struct('type','arbitrary'), 'feas');
fprintf('passivity feasible=%d\n', rp.feas);
```

```
passivity feasible=1
```

The `dwell` struct extends to `'discrete'`, `'periodic'` (`Tbar`), `'minimum'` (`Tbar`), and `'range'` (`[Tmin,Tmax]`), and the system may carry state-multiplicative (`Ac`, `Dwc`), Poisson (`Jf`, `lamf`, `Bdf`), and jump (`J`, `Bd`, `CJ`, `DJ`) data. The multi-mode variant is `mvst_diss_sos_modes`. This routine requires SOSTOOLS and an SDP backend; `S.l2gain` and `mvst_passivity_ct` need only base MATLAB.

7.6 Worked case study: compute an L2 gain exactly, then certify it by SOS

This end-to-end example takes a single non-normal CT system, computes its L2-gain three independent ways, and confirms they agree: the exact bounded-real Riccati engine (`S.l2gain`), the SOS dissipativity dual (`mvst_diss_sos`), and a direct frequency sweep of the transfer function `|C(jw I - A)^{-1}Bw + Dw|`. Agreement across all three is the practical check that the certificate is real.

```
A = [-2 1; 0 -3]; Bw = [1;1]; C = [1 0]; Dw = 0;
S  = mvst.System.ct(A, zeros(2,0), 'Bw', Bw, 'C', C, 'Dw', Dw);
ex = S.l2gain();
sos = mvst_diss_sos(struct('A0',A,'Bw',Bw,'C',C,'Dw',Dw), ...
                    struct('type','l2gain'), struct('type','arbitrary'), 'gamma');
w = logspace(-2,2,20000); G = zeros(size(w));
for k = 1:numel(w); G(k) = abs(C*((1i*w(k)*eye(2)-A)\Bw) + Dw); end
fprintf('exact Riccati gamma = %.6f  (method=%s)\n', ex.gamma, ex.method);
fprintf('SOS-dual     gamma = %.6f  (|diff|=%.2e)\n', sos.gamma, abs(ex.gamma-sos.
    ↪ gamma));
fprintf('freq-sweep  Hinf   = %.6f\n', max(G));
```

```
exact Riccati gamma = 0.666687  (method=ct)
SOS-dual     gamma = 0.666667  (|diff|=2.05e-05)
freq-sweep  Hinf   = 0.666657
```

Interpretation of the run: the exact bounded-real Riccati gain is 0.666687, the SOS dual returns 0.666667 (within 2e-5, the relaxation gap), and the brute-force peak of the frequency response is 0.666657. Three methods that never touch each other's code agree to five digits, so the reported L2-gain is trustworthy. In a design loop one would use `S.l2gain` for the fast exact value and `mvst_diss_sos` when a dwell-time-constrained or multiplicative-noise system takes the exact backend out of reach.

For linear positive systems, dissipativity with a general linear supply is handled by the copositive-LP routine `mvst_pos_supply` (see Positive systems).

7.7 Further dissipativity engines

7.7.1 `mvst_kyp`: operator-domain KYP

`res = mvst_kyp(sys, opts)` solves the Kalman-Yakubovich-Popov problem for the matrix-valued second-moment dynamics, in continuous time, discrete time, or the periodic impulsive setting (`opts.domain` is `'ct'` (default), `'dt'`, or `'hybrid'`). The supply is chosen by `opts.supply` (`'boundedreal'`, `'passivity'`) or given as an arbitrary-sign matrix `opts.M`. The engine solves both the LMI form and the zero-frequency (dual) form, reports which one certifies, and returns a frequency-domain witness when the LMI is infeasible. Both forms are SOSTOOLS programs (primal SDPs with the witness as the primal variable), so every backend works, including `'nosdp'`; the legacy YALMIP formulation is available as `opts.engine = 'yalmip'` (optional external dependency).

```
A = [-1 0.5;0 -2]; Bw = [1;0.5]; C = [1 1]; Dw = 0.2;
r = mvst_kyp(struct('A',A,'Bw',Bw,'C',C,'Dw',Dw),struct('domain','ct','supply','
    ↪ boundedreal'));
fprintf('regular=%d gamma=%.4f certifies=%s\n',r.regular,r.gamma,r.certifies);
```

```
regular=1 gamma=1.5750 certifies=both
```

The bisected level `gamma = 1.5750` equals the exact `mvst_l2gain_ct` value. When the noise-channel reachability hypothesis fails, the engine refuses to conclude (`res.regular = false`).

7.7.2 `mvst_isp_margin_ct`: input-strict-passivity index

`[nu, P] = mvst_isp_margin_ct(A, Bw, C, Dw, Cc, Dwc, opts)` returns the largest `nu` such that the Ito system `dx = (A x + Bw w) dt + sum_l (Cc{l} x + Dwc{l} w) dW_l, z = C x + Dw w` is dissipative for the supply `He(w'z) - nu |w|^2`, by an SOS (constant-certificate) program; `Cc` and `Dwc` are cells (possibly empty). Without noise, `nu` equals the minimum over frequency of the smallest eigenvalue of `G(jw) + G(jw)'` (the convention shared with `mvst_passivity`).

```
nu = mvst_isp_margin_ct([-1 0.5;0 -2],[1;0.5],[1 1],2,{},{});
fprintf('nu=%.4f\n',nu);
```

```
nu=4.0000
```

A positive `nu` certifies input strict passivity; `P` (second output) is the constant storage certificate.

7.7.3 `mvst_qsr_supply`: quadratic supply contract

`[Q,S,R] = mvst_qsr_supply(supply, p, m)` validates and unpacks the general quadratic supply `s(z,w) = [z;w]'[Q S;S' R][z;w]` for an output of dimension `p` and a disturbance of dimension `m`. Pass `'jump'` as a fourth argument to select the jump weights (`supply.Qk`, `Sk`, `Rk`). It is the single validation point used by the SOS dissipativity engines and `mvst.System.dissipativity`.

```
sup = struct('type','qsr','Q',-1,'S',0,'R',4);   % L2-gain supply, gamma = 2
[Q,S,R] = mvst_qsr_supply(sup,1,1);              % -> Q = -1, S = 0, R = 4
```

The sign convention is that of the Dissipativity chapter: dissipativity means `d/dt <P,x> <= s`; the L2-gain supply is `Q = -I`, `S = 0`, `R = gamma^2 I`, and the passivity supply with indices `(nu,rho)` is `Q = -rho I`, `S = I`, `R = -nu I`.

7.7.4 `mvst_hinfnorm`: H-infinity norm with a uniform result

`res = mvst_hinfnorm(sys, opts)` returns the H-infinity norm (mean-square L2-gain) through the class-routed exact backends of `mvst_l2gain`, in one uniform struct: `res.gamma`, `res.certificate` (a storage struct `.S`, `.gamma`, `.residual` where available, `[]` otherwise), `res.class`, `res.method` (the backend that produced the number), and `res.diagnostics` (including `.exact`, false for shooting lower bounds). The object method is `S.hinfnorm()`.

```
r = mvst_hinfnorm(struct('A0',[-1 1;0 -2],'Bw',[1;0],'C',[1 0]));
fprintf('gamma=%.4f class=%s method=%s\n',r.gamma,r.class,r.method);
```

```
gamma=1.0000 class=ct method=mvst_l2gain_ct
```

7.7.5 `mvst_passivity`: routed passivity analysis with indices

`res = mvst_passivity(sys, opts)` tests passivity of a square-port system and returns the input-strict-passivity index `res.nu_isp` (the largest `nu` with supply `He(w'z) - nu |w|^2` dissipative; for an LTI system the minimum over frequency of the smallest eigenvalue of `G(jw) + G(jw)'`), the output-strict index `res.rho_osp`, a flag `res.passive`, a storage certificate, and the method used (`res.method`). The default for continuous-time LTI systems is the exact imaginary-axis test of the positive-real Hamiltonian; noise, mean-square, and dwell-time classes route to the SOS programs. The object method is `S.passivity()`.

```
r = mvst_passivity(struct('A0',[0 1;-1 -1],'Bw',[0;1],'C',[1 0],'Dw',0.5));
fprintf('passive=%d nu=%.4f rho=%.4f method=%s\n',r.passive,r.nu_isp,r.rho_osp,r.
    ↪ method);
```

```
passive=1 nu=0.3333 rho=0.7534 method=hamiltonian
```

7.7.6 `mvst_l2gain_dt` and `mvst_l2gain_dt_mjls`: discrete-time gains

`gam = mvst_l2gain_dt(A, Bw, C, Dw)` returns the H-infinity norm over the unit circle of `x+ = A x + Bw w, z = C x + Dw w` (A Schur), exactly, through the bilinear map to continuous time and the imaginary-axis Hamiltonian bisection. No Control System Toolbox is needed. `[gam, S] = mvst_l2gain_dt_mjls(Ac, Bw, Cc, Dw, Ptr)` returns the coupled mean-square gain of a discrete-time Markov-jump system (cells `Ac`, `Cc` of mode matrices, row-stochastic transition matrix `Ptr`) by the coupled bounded-real Riccati recursion, together with the cell `S` of mode storage matrices as certificate. Both are reached through `mvst_l2gain` and `mvst_hinfnorm` when the system is flagged discrete-time.

```
g  = mvst_l2gain_dt(0.5,1,1,0);                              % 1/(1-0.5)
gm = mvst_l2gain_dt_mjls({0.5,0.4},1,{1,1},0,[0.9 0.1;0.2 0.8]);
fprintf('dt=%.4f  dt-mjls=%.4f\n',g,gm);
```

```
dt=2.0000  dt-mjls=1.9417
```

The Markov-jump value is a lower bound that converges from below as the recursion horizon grows (see `help mvst_l2gain_dt_mjls`).

Chapter 8

Games

This chapter covers the state-feedback zero-sum `H_infinity` game: the exact critical level of `mvst_game_gamma` (method `S.gamegamma`) and the SOS relaxation of `mvst_game_sos`. It corresponds to the book chapter **Differential Games** of the companion monograph. The section below states the game in full so the chapter is self-contained.

8.1 Mathematical setting

The two-player system. The state `x` is driven by a **minimizing** control `u` (mu channels) and a **maximizing** disturbance `w` (mw channels):

$$\mathrm{d}x = (Ax + B_u u + B_w w)\,\mathrm{d}t + \sum_l (A_l x + B_l u)\,\mathrm{d}W_l.$$

The toolbox carries `A` (`A0`), the control input `Bu`, the disturbance input `Bw`, the state weight `Q`, and the control weight `Ru`.

The minimax payoff. With `Q >= 0`, `Ru > 0`, and a disturbance penalty `gamma^2`, the players share the quadratic cost, linear in the joint second moment `X`,

$$J(u,w) = \int_0^\infty \Big(\langle Q, X_1\rangle + \langle R_u, X_{22}\rangle - \gamma^2\langle I, X_{33}\rangle\Big)\,\mathrm{d}t = \mathbb{E}\int_0^\infty \big(\|z\|^2 - \gamma^2\|w\|^2\big)\,\mathrm{d}t,$$

where `z = [Q^{1/2} x; R_u^{1/2} u]` is the performance output and `X_{22}, X_{33}` are the control and disturbance second-moment blocks. The minimizer picks `u` to reduce `J`, the maximizer picks `w` to increase it. The negative sign on the `w`-block encodes the opposing objective. The game value is the saddle

$$J^\circ = \min_u \max_w J(u,w).$$

Isaacs condition. The value is well defined (min-max equals max-min, a saddle point exists) when **Isaacs' condition** holds: the bottom-right Isaacs block of the augmented weight,

$$\mathcal{G} = \begin{bmatrix} R_u & 0 \\ 0 & -\gamma^2 I \end{bmatrix} \qquad (\mathcal{G}_{uu} = R_u \succ 0, \mathcal{G}_{ww} = -\gamma^2 I \prec 0),$$

has the saddle inertia that lets the order of min and max be exchanged. `G` is **indefinite** (positive on the control block, negative on the disturbance block); this indefiniteness is what separates the game from ordinary optimal control.

The game generalized Riccati equation (game GRE). The saddle is characterized by a stabilizing Hermitian solution `P >= 0` of the indefinite (Hamilton-Jacobi-Isaacs) Riccati equation

$$\mathrm{He}(A^*P) + Q - PB_uR_u^{-1}B_u^*P + \tfrac{1}{\gamma^2}PB_wB_w^*P = 0,$$

whose defining feature is the `+ (1/gamma^2)` disturbance term (opposite in sign to the control term). The saddle policies are the two central gains

$$K_u = -R_u^{-1}B_u^*P \quad \text{(optimal control)}, \qquad K_w = \tfrac{1}{\gamma^2}B_w^*P \quad \text{(worst-case disturbance)}.$$

The bounded-real interpretation of `H_infinity`. As `gamma` decreases toward a critical level `gamma*`, the `+ (1/gamma^2)` term grows and the stabilizing solution of the game GRE ceases to exist. That critical level is exactly the closed-loop bounded-real (`H_infinity`) level: the smallest disturbance attenuation the controller can guarantee. For `gamma` $>$ `gamma*` the game GRE has a stabilizing `P >= 0` (the controller wins); for `gamma` $<$ `gamma*` it does not. The toolbox finds `gamma*` by **bisecting** on `gamma`, at each trial testing whether the game GRE (or its SOS certificate) is solvable. **Standing assumptions:** `(A,B_u)` stabilizable, `Q >= 0`, `R_u` $>$ `0`.

8.2 The bounded-real interpretation, in the toolbox

The critical level `gamma*` is what `S.gamegamma` returns. A feasibility solve at a fixed `gamma` (`mode = 'feas'`) certifies the bounded-real property at that level: feasible above `gamma*`, infeasible below it. The worked example below shows exactly this crossover.

8.3 Exact level: `S.gamegamma`

`S.gamegamma` computes `gamma*` by gamma-bisection over the game Riccati of the resolved class. Supply the state weight `Q`, control weight `R`, control input `B`, and disturbance input `Bw` through the factory:

```
S = mvst.System.ct([0 1;-1 -0.5], [0;1], 'Q', eye(2), 'R', 1, 'Bw', [1;0]);
r = S.gamegamma();
fprintf('gamma* = %.6f  method=%s\n', r.gammastar, r.method);
```

```
gamma* = 1.293481  method=ct
```

`res.gammastar` is the critical level, `res.P` the settled dual(s), and `res.method` the resolved class. The backends are `mvst_game_gamma_ct`, `_ct_mjls`, `_ct_dwell`, and `_ct_poisson`; the sampled-data game shares the `ct` backend (use `OPTS.class = 'ct'`).

8.4 SOS relaxation: `mvst_game_sos`

`mvst_game_sos(sys, game, dwell, mode)` returns the critical level and central gains over every dwell-time type of the chapter.

Caveat. The book's game Riccati inequality is not jointly convex in the dual variable (the Isaacs block `G` is indefinite, so the generalized Schur complement is rational in `P` and paired with the saddle-inertia condition), so it cannot be handed to an SDP solver as stated. This routine instead solves the sufficient convex relaxation from the classical bounded-real change of variables. For the separated continuous-time LTI game (zero cross term) the relaxation is **exact** and reproduces the true `gamma*`; for the timer-dependent / dwell-time certificates it returns a certified **upper bound** that tightens as the certificate degree grows. The exact per-class solvers above always return the exact `gamma*`.

The example below solves the same system by the arbitrary-dwell-time SOS program and compares to the exact level:

```
Ag=[0 1;-1 -0.5]; Bu=[0;1]; Bw=[1;0]; Q=eye(2); Ru=1;
rg = mvst_game_sos(struct('A0',Ag,'Bu',Bu,'Bw',Bw), ...
                   struct('Q',Q,'Ru',Ru), struct('type','arbitrary'), 'gamma');
ex = mvst_game_gamma(struct('A',Ag,'Bu',Bu,'Bw',Bw,'Q',Q,'Ru',Ru));
fprintf('SOS gamma=%.6f  exact gamma*=%.6f\n', rg.gamma, ex.gammastar);
```

```
SOS gamma=1.293300  exact gamma*=1.293481
```

The SOS value (1.293300) matches the exact level (1.293481) to about 2e-4, as expected for this separated game. The `dwell` struct extends to `'discrete'`, `'periodic'`, `'minimum'`, and `'range'`, and the system may carry state-multiplicative and Poisson jump data. This routine requires SOSTOOLS and an SDP backend; `S.gamegamma` needs only base MATLAB.

8.5 Worked case study: the critical level and the bounded-real crossover

This end-to-end example takes an undamped-plus-feedback oscillator, computes the critical level exactly, extracts both saddle gains from the settled game GRE solution, and then demonstrates the bounded-real interpretation directly: a feasibility solve **below** `gamma*` fails and one **above** `gamma*` succeeds.

```
A = [0 1; -2 -1]; Bu = [0;1]; Bw = [1;0]; Q = eye(2); Ru = 1;
S  = mvst.System.ct(A, Bu, 'Q', Q, 'R', Ru, 'Bw', Bw);
ex = S.gamegamma();
sos = mvst_game_sos(struct('A0',A,'Bu',Bu,'Bw',Bw), ...
                    struct('Q',Q,'Ru',Ru), struct('type','arbitrary'), 'gamma');
P  = ex.P;
Ku = -Ru\(Bu'*P);           % optimal control gain
Kw = (1/ex.gammastar^2)*(Bw'*P);  % worst-case disturbance gain
fprintf('exact gamma* = %.6f   SOS gamma = %.6f   |diff| = %.2e\n', ...
        ex.gammastar, sos.gamma, abs(ex.gammastar-sos.gamma));
fprintf('Ku = %s\nKw = %s\n', mat2str(Ku,5), mat2str(Kw,5));
for g = [0.9 1.1]*ex.gammastar
    rf = mvst_game_sos(struct('A0',A,'Bu',Bu,'Bw',Bw), ...
              struct('Q',Q,'Ru',Ru,'gamma',g), struct('type','arbitrary'), 'feas');
    fprintf('  gamma=%.4f (=%.2f*gamma*)  feas=%d\n', g, g/ex.gammastar, rf.feas);
end
```

```
exact gamma* = 1.331323   SOS gamma = 1.336055   |diff| = 4.73e-03
Ku = [-2.7763 -2.4252]
Kw = [3.2336 1.5664]
  gamma=1.1982 (=0.90*gamma*)  feas=0
  gamma=1.4645 (=1.10*gamma*)  feas=1
```

Interpretation of the run: the exact critical level is `gamma* = 1.331323`, and the SOS relaxation gives 1.336055, a certified upper bound within 5e-3. The two central gains follow from the settled game GRE solution `P`: `Ku` is the minimizing controller, `Kw` the direction the disturbance would push. The last two lines are the bounded-real interpretation made concrete: at `gamma = 0.90 gamma*` (below the critical level) the game is infeasible (`feas=0`, no controller can attenuate that hard), while at `gamma = 1.10 gamma*` (above it) the game is feasible (`feas=1`). This is exactly what the bisection in `S.gamegamma` certifies: it squeezes `gamma` down to the boundary between these two regimes.

8.6 Further game engines

These engines are called directly as `mvst_*` functions; the object method `S.gamegamma` covers the critical level only.

8.6.1 `mvst_game_average`: ergodic zero-sum value

`res = mvst_game_average(sys, opts)` returns the long-run average value `rho` of the zero-sum game under a persistent forcing `sys.W` (a constant covariance, or a handle `@(t)` with period `sys.Tp`), at a level `sys.gamma` above the critical level. The homogeneous part is the H-infinity game dual; the value is linear in the forcing.

```
sys = struct('A',[-1 0.5;0 -2],'Bu',[1;0.5],'Bw',[0.3;0.6],'Q',eye(2),'Ru',1);
g = mvst_game_gamma(sys); sys.gamma = 1.3*g.gammastar; sys.W = eye(2);
r = mvst_game_average(sys);
fprintf('gammastar=%.4f rho=%.4f residual=%.1e\n',g.gammastar,r.rho,r.residual);
```

```
gammastar=0.3840 rho=0.7495 residual=3.7e-10
```

`res.Pinf` is the homogeneous game dual and `res.Ku`, `res.Kw` the saddle gains.

8.6.2 `mvst_game_mixed`: mixed H2 / H-infinity design

`res = mvst_game_mixed(sys, opts)` solves the mixed design as a two-player nonzero-sum Nash game: the designer carries the H2 control cost, the fictitious disturbance carries the bounded-real cost at level `sys.gamma` (required). The two coupled Riccati legs are solved by a Gauss-Seidel iteration. As `gamma` grows, the solution tends to the pure H2 solution and `Kw` to zero.

```
sys = struct('A',[-1 0.5;0 -2],'Bu',[1;0.5],'Bw',[0.3;0.6],'Q',eye(2),'Ru',1,'Su',
    ↪ zeros(2,1));
sys.gamma = 2*0.3840;                          % twice the critical level
r = mvst_game_mixed(sys);
fprintf('iters=%d residual=%.1e h2cost=%.4f\n',r.iters,r.residual,r.h2cost);
```

```
iters=8 residual=1.1e-10 h2cost=0.7082
```

The gains are `res.Ku = [-0.4606 -0.2072]` and `res.Kw = [0.1978 0.2655]`. Below the critical level the iteration does not converge.

8.6.3 `mvst_game_nash`: N-player Nash equilibrium

`res = mvst_game_nash(sys, opts)` solves the coupled generalized Riccati system of an N-player game by a Gauss-Seidel sweep. `sys.Bu`, `sys.Q` are 1-by-N cells and `sys.R{i}{j}` is player `i`'s weight on control `j`. A two-player zero-sum instance reproduces the H-infinity saddle.

```
sys = struct('A',[-1 0.5;0 -2],'Bu',{{[1;0.5],[0.3;0.6]}},'Q',{{eye(2),-eye(2)}},'R'
    ↪ ,{{{1,-4},{-1,4}}});
r = mvst_game_nash(sys);
fprintf('iters=%d residual=%.1e cost=%s\n',r.iters,r.residual,mat2str(r.cost,4));
```

```
iters=3 residual=6.9e-11 cost=[0.665 -0.665]
```

`res.K{i}` and `res.P{i}` are the equilibrium gains and duals; the zero-sum costs are opposite, as expected.

8.6.4 `mvst_game_sd_held` and `mvst_game_gamma_sd_held`: held-input sampled-data game

`out = mvst_game_sd_held(A,Bu,Bw,Q,Ru,gam,Tp,Dwc,Nmax,tol)` solves by backward iteration the periodic HJI Riccati equation of the zero-sum game on a sampled-data plant whose control is held between samples of period `Tp` (`Dwc` is the optional disturbance-multiplicative noise matrix, `[]` for none). `[gstar,lo] = mvst_game_gamma_sd_held(A,Bu,Bw,Q,Ru,Tp,Dwc)` bisects the critical level, returning the feasible upper end and the infeasible lower end of the final bracket.

```
A=[-1 0.5;0 -2]; Bu=[1;0.5]; Bw=[0.3;0.6]; Q=eye(2); Ru=1; Tp=0.2;
[gstar,lo] = mvst_game_gamma_sd_held(A,Bu,Bw,Q,Ru,Tp,[]);
o = mvst_game_sd_held(A,Bu,Bw,Q,Ru,1.2*gstar,Tp,[]);
fprintf('gstar=%.4f ok=%d reason=%s Ku=%s\n',gstar,o.ok,o.reason,mat2str(o.Ku,4));
```

```
gstar=0.3839 ok=1 reason=converged Ku=[-0.4623 -0.2479]
```

`out.ok` is true when the iteration converged with the Isaacs conditions in force; `out.reason` otherwise names the failure (`'escape'`, `'isaacs_flow'`, `'isaacs_jump'`, `'blowup'`, `'maxiter'`). `out.Ku` is the sample-time gain.

Chapter 9

Dissipative synthesis

This chapter covers dissipative state-feedback synthesis (`mvst_dissyn_sos`, method `S.dissyn`) and its output-feedback sibling (`mvst_dof_sos`). It corresponds to the book chapter **Dissipativity-Based Synthesis** of the companion monograph. The section below states the synthesis problem in full so the chapter is self-contained.

9.1 Mathematical setting

The problem. Given the ported system

$$\mathrm{d}x = (Ax + B_u u + B_w w)\,\mathrm{d}t + \sum_l (A_l x + B_l u)\,\mathrm{d}W_l, \qquad z = Cx + D_u u + D_w w,$$

find a state-feedback gain `K` (and, when the system jumps, a jump gain `K_J` acting on the pre-jump state) so that the closed loop under

$$u = Kx, \qquad \dot{x} = (A + B_u K)x + B_w w,$$

is **dissipative** with respect to a prescribed supply, over a chosen dwell-time regime. For the `L2`-gain supply this means: render the closed loop mean-square stable and drive its `L2`-gain below a level `gamma`, then minimize `gamma`.

Why it needs convexifying. The dissipation certificate of the analysis chapter, written for the closed loop, is the inequality (schematically)

$$\mathrm{He}\big((A + B_u K)^* P\big) + (\text{supply and noise terms}) \preceq 0,$$

which is **bilinear** in the storage matrix `S = -P` and the unknown gain `K`. Bilinear matrix inequalities are not convex and cannot be handed to an SDP solver directly.

The convexifying change of variables. Following the chapter, one congruences the closed-loop inequality by the inverse storage matrix and substitutes new variables

$$W := S^{-1}, \qquad Y := KW, \qquad Y_J := K_J W^{-},$$

after which the design inequality becomes **affine** in `(W, Y, Y_J)`, hence an LMI (a "synthesis block"). With the shorthands `V_l = A_l W + B_l Y` and `G_0 = C W + D_u Y`, the flow synthesis block has the schematic Schur form

$$\Phi_f = \begin{bmatrix} -\dot{W} + \mathrm{He}(AW + B_u Y) & B_w - G_0^* \Pi_2 & \mathrm{row}_l[V_l^*] & (NG_0)^* \\ * & -\Pi_3 - \mathrm{He}(D_w^* \Pi_2) & \mathrm{row}_l[\cdot] & (ND_w)^* \\ * & * & -I \otimes W & 0 \\ * & * & * & -I_\nu \end{bmatrix} \preceq 0,$$

with a matching jump block `Phi_j` at resets, where `Pi = [Pi_1 Pi_2; Pi_2^* Pi_3]` is the port weight pricing `(z; w)`. The **convexifiability** condition is `Pi_1 <= 0`; factoring `-Pi_1 = N^{*} N` with `N` of full row rank (`nu` rows), the fourth (performance) row and column above carry the output-strict channel `(N G_0)^{*}`, `(N D_w)^{*}` closed by `-I_nu`. For the L2-gain supply `Pi_1 = -I_pz` (so `N = I_pz`, `nu = pz`); for output-strict passivity `N = sqrt(rho) I`, and for a non-output-strict supply `nu = 0` and the fourth row and column drop out. Once feasible, the gains are recovered by

$$K = YW^{-1}, \qquad K_J = Y_J(W^{-})^{-1}.$$

The dwell-time scenarios. The certificate `W(tau)` is allowed to depend on a timer `tau` (the clock since the last jump), which is what makes causal dwell-time synthesis possible. Four regimes are assembled:

- **arbitrary**: fixed jump sequence or CT LTI, constant `W`;
- **periodic**: inter-jump time `= Tp`, flow LMI on `[0, Tp]` plus a jump LMI;
- **minimum**: inter-jump time `>= Tmin`, flow LMI on `[0, Tmin]` plus a frozen-tail LMI (`Wdot = 0`) plus a jump LMI;
- **range**: inter-jump time in `[Tmin, Tmax]`, flow LMI on `[0, Tmax]` plus a jump LMI parametrized over the range.

Standing assumptions: `(A, B_u)` stabilizable; the port weight is convexifiable (`Pi_1 <= 0`), which holds for the L2-gain and passivity supplies; `W(tau) > 0` throughout.

9.2 `S.dissyn`: dissipative state feedback

`S.dissyn(supply, dwell, mode)` searches (SOSTOOLS + SeDuMi) for a state-feedback gain `K(tau)` (and a jump gain `K_J`) rendering the closed loop dissipative with respect to `supply` over the dwell-time regime given by `dwell`, using exactly the `W = S^{-1}`, `Y = K W` convexification above. `mode` is `'gamma'` (optimize the supply level) or `'feas'` (certify feasibility at a fixed level).

Arguments default to the L2-gain supply and the arbitrary-dwell-time regime, so the one-line form synthesizes the minimal-L2-gain state feedback:

```
S = mvst.System.ct([0 1;-2 -1], [0;1], 'Bw', eye(2), 'C', [1 0;0 0]);
rs = S.dissyn();
fprintf('dissyn gammastar=%.6f  feas=%d\n', rs.gammastar, rs.feas);
```

```
dissyn gammastar=0.046472  feas=1
```

`res.gammastar` is the certified supply level, `res.feas` the feasibility flag, `res.K` the gain function `@(tau) -> K`, `res.KJ` the jump gain (empty when there is no scheduled jump), `res.W` the synthesis certificate `@(tau) -> W`, and `res.S` the storage `@(tau) -> S = W^{-1} = -P`.

9.2.1 The validation gate

This subsection is the one to read. An SDP solver reports "feasible" whenever it finds a point satisfying the constraints **to its own numerical tolerance**, and those tolerances can be loose enough to certify garbage: a certificate that is barely indefinite, an LMI residual that is small only because the problem is badly scaled, or a gain that does not in fact stabilize the true (unrelaxed) closed loop. A raw `feas = true` from the SDP is therefore not, by itself, trustworthy.

For this reason `feas = true` is **never** returned on the strength of the SOS slack alone. Before returning, the routine recomputes the actual certificate and LMI blocks at the returned `(W, Y, Y_J)` and runs four gates:

1. **Certificate positive definiteness.** The certificate `W(tau)` is checked to be PD on the timer grid: `min eig W(tau) >= epsW/2` at every sample. A near-singular `W` (which inverts into an enormous storage matrix) is rejected.
2. **LMI residual check.** The flow block (and the frozen-tail block for the minimum-dwell-time regime, and the jump block at resets) is recomputed at the returned variables and checked to be negative semidefinite within `restol`. This catches the case where the solver's internal residual was optimistic.
3. **Closed-loop stability.** The recovered gain `K = Y W^{-1}` is fed back and the closed loop's exact lifted spectral abscissa is computed; it must be negative (mean-square stable). A gain that satisfies the LMI numerically but destabilizes the true closed loop is rejected here.
4. **Achieved-vs-certified re-analysis.** For the exact continuous-time L2-gain case, the synthesized closed loop is handed back to the exact bounded-real backend `mvst_l2gain_ct`, and the **achieved** gain must be no worse than the certified level, within `gaptol`. This is the decisive check: the certificate is only accepted if an independent exact solver confirms the claimed performance.

A gate failure demotes the level to infeasible, so the gamma-bisection converges to a certified level; if nothing passes, `res.feas = false` and `res.info.gate` names the failing gate. In short, `feas = true` now means the synthesized gain has been re-analyzed by the exact backend and confirmed, not merely that an SDP reported a positive slack. The gate diagnostics live in `res.info.gate`: `.ok`, `.reason`, `.mineigW`, `.condW`, `.abscissa`, and `.achieved`. The standalone check `mvst_dissyn_sos('selftest')` re-runs this re-analysis across a battery.

The `supply` struct accepts the same `'l2gain'` / `'passivity'` types as the analysis routine (see Dissipativity), and the `dwell` struct the same `'arbitrary'` / `'periodic'` / `'minimum'` / `'range'` regimes.

9.3 Worked case study: synthesize, inspect the gate, close the loop, re-analyze

This end-to-end example synthesizes a minimal-L2-gain state feedback for an oscillator whose open-loop gain is high, then opens up the validation gate to see what was checked, reconstructs the gain, closes the loop, and re-analyzes it with the independent exact backend. The point is to show that the certified level is not taken on faith.

```
A = [0 1; -2 -1]; Bu = [0;1]; Bw = eye(2); C = [1 0; 0 0]; Dw = [0 0; 0 1];
S  = mvst.System.ct(A, Bu, 'Bw', Bw, 'C', C, 'Dw', Dw);
rs = S.dissyn();
g  = rs.info.gate;
fprintf('certified gammastar = %.6f   feas=%d   slack=%.2e\n', ...
        rs.gammastar, rs.feas, rs.slack);
fprintf('gate.ok=%d (%s)\n', g.ok, g.reason);
fprintf('  (1) min eig W  = %.4f   cond(W) = %.1f\n', g.mineigW, g.condW);
fprintf('  (3) closed-loop spectral abscissa = %.4f\n', g.abscissa);
fprintf('  (4) achieved gain = %.6f  (<= certified %.6f)\n', g.achieved, rs.
    ↪ gammastar);
K   = rs.K(0);
Acl = A + Bu*K;
fprintf('gain K = %s\n', mat2str(K,5));
fprintf('eig(Acl) = %s\n', mat2str(sort(eig(Acl)),5));
reanalysis = mvst_l2gain_ct(Acl, Bw, C, Dw);
openloop   = mvst_l2gain_ct(A,   Bw, C, Dw);
fprintf('open-loop L2 gain    = %.6f\n', openloop);
fprintf('independent re-check = %.6f\n', reanalysis);
```

```
certified gammastar = 1.000100   feas=1   slack=3.87e-05
gate.ok=1 (pass)
  (1) min eig W  = 4.8554   cond(W) = 835.2
  (3) closed-loop spectral abscissa = -10.9409
  (4) achieved gain = 1.000062  (<= certified 1.000100)
gain K = [-796.51 -9.9409]
eig(Acl) = [-5.4705-27.723i;-5.4705+27.723i]
open-loop L2 gain    = 1.565858
independent re-check = 1.000062
```

Interpretation of the run: the synthesizer certifies `gamma* = 1.000100`, down from the open-loop gain of 1.565858, so feedback improves the attenuation. The gate then confirms the certificate is real: (1) `W` is comfortably positive definite (smallest eigenvalue 4.86, condition number 835), (3) the closed loop is deeply stable (spectral abscissa -10.94), and (4) the achieved gain recomputed by the exact backend is 1.000062, no worse than the certified level. Reconstructing `K = Y W^{-1}` and closing the loop independently gives a stable `Acl` (a well-damped complex pair), and re-analyzing it from scratch reproduces 1.000062: the certified number and the true achieved number agree. Note the large gain magnitude (about `-800` on the first state); pushing `gamma` to the plant's floor demands high feedback authority, a real trade-off the certified number does not hide.

9.4 `mvst_dof_sos`: dynamic output feedback

`mvst_dof_sos(sys, dwell, opts)` is the output-feedback counterpart. It belongs to the dynamic output-feedback development (treated separately from the state-feedback chapter above), and synthesizes a timer-dependent dynamic output-feedback controller reconstructed as `(A_c, B_c, C_c, D_c)`. Its signature is deliberately shorter than the other SOS routines (no separate supply / cost argument, no mode selector): the program is a fixed convex bracket in the timer, solved without any inverse of the certificate. The example uses the scalar timer-dependent plant of the output-feedback example under a periodic dwell-time:

```
sys.a0 = 1.0; sys.a1 = 0.4; sys.sig = 0.5; sys.mu = 0.5;
sys.B0 = 1; sys.Cy = 1; sys.J0 = 0.5; sys.BJ = 1; sys.CJ = 1;
opts.alpha = 1e-3; opts.eps = 1e-3;
out = mvst_dof_sos(sys, struct('type','periodic','T',1.0,'deg',2), opts);
fprintf('periodic DOF solved=%d  max lam(M)=%+.3e  min lam(Om)=%+.3e\n', ...
        out.feas, out.maxM, out.minOm);
```

```
periodic DOF solved=1  max lam(M)=-3.416e-01  min lam(Om)=+5.925e-01
```

`out.feas` is the feasibility flag; `out.maxM` (the maximum eigenvalue of the flow bracket, negative here) and `out.minOm` (the minimum eigenvalue of the coupling block, positive here) are the two certificate margins verified over the whole dwell-time interval, and `out.K` carries the reconstructed controller `(A_c, B_c, C_c, D_c)` and jump gains. The `dwell` struct extends to `'minimum'` and `'range'`.

Both routines require SOSTOOLS and an SDP backend.

Chapter 10

Positive systems

This chapter covers the linear positive-systems family: analysis and synthesis by copositive linear programs. It corresponds to the **Linear Positive Systems** chapter of the companion monograph. Unlike the SOS layer, this family uses `linprog` and so needs the **Optimization Toolbox** rather than an SDP backend.

The sections below are written to stand on their own: every problem is stated as a display before the code that solves it, so the chapter is usable without the book.

10.1 The mathematical setting

A **linear positive system** is a linear system whose state stays in the nonnegative orthant: if the initial state is nonnegative, every future state is nonnegative. Two entrywise matrix notions organize the whole family.

A square matrix M is **nonnegative**, written $M \geq 0$, when all its entries are nonnegative, and a matrix A is **Metzler** when its off-diagonal entries are nonnegative:

$$M \geq 0 \iff M_{ij} \geq 0 \text{ for all } i, j, \qquad A \text{ Metzler} \iff A_{ij} \geq 0 \text{ for all } i \neq j.$$

For the continuous-time system $\dot{x} = Ax$ with resets $x(t_k^+) = Jx(t_k^-)$, positivity is characterized entrywise:

$$e^{At} \geq 0 \text{ for all } t \geq 0 \iff A \text{ is Metzler}, \qquad Jx \geq 0 \text{ for all } x \geq 0 \iff J \geq 0.$$

So a system is positive exactly when its drift is Metzler at every time and every reset map is nonnegative. The discrete-time content is carried by the nonnegative reset J and by spectral-radius conditions such as $\rho(Je^{A\bar{T}}) < 1$; there is no separate discrete state equation.

10.1.1 Linear copositive Lyapunov functions

On the nonnegative orthant a **linear** functional already serves as a Lyapunov function. For a Metzler A the following are equivalent: A is Hurwitz; $-A^{-1} \geq 0$; there is a vector $d > 0$ with $Ad < 0$; and there is a vector $p > 0$ with

$$A^\top p < 0, \qquad \text{so that} \qquad V(x) = p^\top x, \quad \dot{V} = (A^\top p)^\top x < 0 \text{ for } 0 \neq x \geq 0.$$

The function $V(x) = p^\top x$ is a **linear copositive Lyapunov function**: it is positive on the orthant and decreasing along the dynamics. Its existence is a **linear program** in the nonnegative weight p.

10.1.2 Why linear programs replace semidefinite programs

On the cone of positive semidefinite matrices the certificates are quadratic ($V(X) = \langle P, X\rangle$) and their existence is a semidefinite program. On the nonnegative orthant everything collapses one level: the quadratic certificate becomes the **linear** functional $p^\top x$, the LMIs become linear inequalities, and the trace norm becomes the 1-norm. The positive-systems theory is exactly the diagonal slice of the monograph's matrix-valued theory, and on that slice a semidefinite program becomes a linear program. That is the reason this family calls `linprog` and not an SDP solver.

The engines read the drift and I/O data at the top level of a flat struct (`sys.A` a matrix for a single mode, a cell of drifts for a multi-mode system), not through the `.modes` wrapper. The object method `S.pos(what, opts)` bridges a `mvst.System` to that contract; the examples below call the `mvst_pos_*` functions directly, matching the book's example scripts.

10.2 Stability: `mvst_pos_stability`

The continuous-time stability certificate is the copositive linear program

$$\text{find } p \geq \mathbf{1} \quad \text{such that} \quad A^\top p \leq -\varepsilon \mathbf{1},$$

with a small margin $\varepsilon > 0$ and $\mathbf{1}$ the all-ones vector (the lower bound $p \geq \mathbf{1}$ normalizes the strictly positive p). A Metzler-Hurwitz drift is certified:

```
r = mvst_pos_stability(struct('A',[-1 0;3 -1]), struct('class','ct'));
fprintf('stable=%d\n', r.stable);
disp('p='); disp(r.p);
```

```
stable=1
p=
    3.0000
    1.0000
```

The solver returns $p = (3.000001, 1)^\top > 0$ (its printed values round to $(3, 1)$), and then $A^\top p = (-10^{-6}, -1)^\top$, strictly negative entrywise by the LP margin $\varepsilon = 10^{-6}$, so $V(x) = 3x_1 + x_2$ certifies stability. With the exactly rounded $p = (3, 1)^\top$ the first entry is $A^\top p|_1 = -3 + 3 \cdot 1 = 0$, which meets the constraint only non-strictly; the tiny shift the solver adds is exactly that margin. The discrete-time analogue, selected with `opts.class = 'dt'`, tests a nonnegative Schur transition matrix M through

$$\text{find } p \geq \mathbf{1} \quad \text{such that} \quad (M^\top - I)p \leq -\varepsilon \mathbf{1} \quad \Longleftrightarrow \quad M^\top p < p.$$

10.3 Dwell-time stability

When flow and reset are combined, stability holds only in a dwell-time window and the certificate carries the **matrix exponential** of the flow over one dwell-time period.

10.3.1 Impulsive maximum dwell-time

Take an unstable Metzler flow caught by a contractive nonnegative reset. Over one period the monodromy is $Je^{A\bar{T}}$; it is Schur only while the dwell-time $\bar{T}$ stays below a maximum $\bar{T}^\star$. Below that threshold the copositive LP

$$p \geq \mathbf{1}, \qquad A^\top p \geq \varepsilon \mathbf{1}, \qquad \left(Je^{A\bar{T}}\right)^\top p \leq p - \varepsilon \mathbf{1}$$

is feasible: $V = p^\top x$ grows along the expanding flow but the reset more than compensates over each cycle. With the book's running data $A = \left(\begin{smallmatrix} 0.5 & 1 \\ 0 & 0.5 \end{smallmatrix}\right)$, $J = \left(\begin{smallmatrix} 0.1 & 0.2 \\ 0 & 0.1 \end{smallmatrix}\right)$, the exact threshold is $\bar{T}^\star = 2\ln 10 \approx 4.605$:

```
A = [0.5 1; 0 0.5]; J = [0.1 0.2; 0 0.1];
r = mvst_pos_stability(struct('A',A,'J',J,'Tbar',1.5), ...
        struct('class','impulsive','threshold',true));
fprintf('at Tbar=1.5 stable=%d  Tstar=%.4f\n', r.stable, r.Tstar);
```

```
at Tbar=1.5 stable=1  Tstar=4.6052
```

`opts.threshold` also bisects the LP feasibility boundary and returns it in `res.Tstar`, matching the analytic $2\ln 10$. (When the flow is Hurwitz the routine solves instead the constant-dwell-time slice $A^\top p \le -\varepsilon\mathbf{1}$, $(Je^{A\bar{T}})^\top p \le p - \varepsilon\mathbf{1}$.)

10.3.2 Switched minimum dwell-time

For a positive switched system $\dot{x} = A_\sigma x$ with Metzler-Hurwitz modes that share no common linear copositive Lyapunov function, stability is recovered above a **minimum** dwell-time. The mode-dependent certificate uses one weight per mode:

$$p_i \ge \mathbf{1}, \qquad A_i^\top p_i \le -\varepsilon\mathbf{1}, \qquad e^{A_i^\top T_{\min}} p_j \le p_i - \varepsilon\mathbf{1} \quad \text{for each admissible } i \to j.$$

The two shear modes below are each Hurwitz, but $A_1 + A_2$ has an unstable eigenvalue, so fast switching diverges:

```
A1 = [-1 0; 3 -1]; A2 = [-1 3; 0 -1];
r = mvst_pos_stability(struct('A',{{A1,A2}}), ...
        struct('class','switched','dwell','min','Tmin',2.0,'threshold',true));
fprintf('stable=%d  Tstar=%.4f\n', r.stable, r.Tstar);
disp('p1='); disp(r.p{1}); disp('p2='); disp(r.p{2});
```

```
stable=1  Tstar=1.6221
p1=
    3.0000
    1.0000
p2=
    1.0000
    3.0000
```

At $T_{\min} = 2$ the program is feasible; the bisected threshold $\bar{T}^\star \approx 1.62$ is the smallest dwell-time at which the copositive LP stays feasible. Pass `opts.trans` (a $K \times 2$ list of ordered transitions $[i, j]$) to restrict the admissible switches; the default is all $i \ne j$.

10.3.3 Markov-jump systems

A positive Markov-jump system with generator $\Pi = (\pi_{ij})$ is certified by the **coupled** copositive program

$$A_i^\top p_i + \sum_j \pi_{ij} p_j \le -\varepsilon\mathbf{1}, \qquad p_i \ge \mathbf{1}, \qquad i = 1, \dots, N.$$

```
A1 = [-2 0.5; 0.3 -2]; A2 = [-1.5 0.2; 0.6 -1.8]; Pi = [-1 1; 1 -1];
r = mvst_pos_stability(struct('A',{{A1,A2}},'Pi',Pi), struct('class','mjls'));
fprintf('stable=%d\n', r.stable);
```

```
stable=1
```

The generator couples the mode weights: unlike the switched case each mode gets its own p_i, but the rows are linked through $\sum_j \pi_{ij} p_j$.

10.4 Induced gains: `mvst_pos_gain`

For a stable positive LTI system

$$\dot{x} = Ax + Ew, \qquad z = Cx + Fw,$$

with A Metzler-Hurwitz and $E, C, F \geq 0$, the induced gains are attained at constant inputs and are computed by linear programs.

10.4.1 The L1 gain

The induced $L_1 \times \ell_1$ gain is the worst-case amplification of the integrated (total-mass) signals,

$$\gamma_1 = \sup_{w \neq 0} \frac{\int_0^\infty \mathbf{1}^\top z(t)\, dt}{\int_0^\infty \mathbf{1}^\top w(t)\, dt},$$

and it is the value of the copositive linear program

$$\min_{\gamma, \lambda \geq 0} \gamma \quad \text{s.t.} \quad A^\top \lambda + C^\top \mathbf{1} \leq 0, \qquad E^\top \lambda + F^\top \mathbf{1} \leq \gamma \mathbf{1}.$$

The optimum is the vertex $\lambda^\star = -(A^\top)^{-1} C^\top \mathbf{1}$, giving $\gamma_1 =$ max-channel of $E^\top \lambda^\star + F^\top \mathbf{1}$. In the **scalar (SISO)** case this equals the DC gain $C(-A)^{-1}E$ (the area under the nonnegative impulse response); for a MIMO system $C(-A)^{-1}E$ is a matrix and γ_1 is its largest **column** channel, not a single scalar.

```
r = mvst_pos_gain(struct('A',[-0.9 0;0.5 -0.6],'E',[1;0],'C',[1 1]), ...
                  struct('norm','L1'));
fprintf('L1 gain = %.6f  gammaDC=%.6f\n', r.gamma, r.gammaDC);
```

```
L1 gain = 2.037037  gammaDC=2.037037
```

The LP value and the closed-form DC gain agree, as they must for a positive system. `res.weights` returns the dual certificate $\lambda^\star$.

10.4.2 The Linf (peak-to-peak) gain

The induced $L_\infty \times \ell_\infty$ (peak-to-peak) gain amplifies the peak amplitudes,

$$\gamma_\infty = \sup_{w \neq 0} \frac{\sup_t \|z(t)\|_\infty}{\sup_t \|w(t)\|_\infty},$$

and is the value of the dual (primal-tube) copositive program

$$\min_{\gamma, \xi \geq 0} \gamma \quad \text{s.t.} \quad A\xi + E\mathbf{1} \leq 0, \qquad C\xi + F\mathbf{1} \leq \gamma \mathbf{1},$$

with optimum $\xi^\star = (-A)^{-1} E\mathbf{1}$ and $\gamma_\infty = \max(C\xi^\star + F\mathbf{1})$. In the **scalar (SISO)** case this is again the DC gain, so $\gamma_1 = \gamma_\infty = C(-A)^{-1}E$; for a MIMO system γ_∞ is the largest **row** channel of the DC-gain matrix, so in general γ_1 (worst column) and γ_∞ (worst row) differ and coincide only in the scalar case shown here:

```
r = mvst_pos_gain(struct('A',[-0.9 0;0.5 -0.6],'E',[1;0],'C',[1 1]), ...
                  struct('norm','Linf'));
fprintf('Linf gain = %.6f\n', r.gamma);
```

```
Linf gain = 2.037037
```

Select the norm with `opts.norm = 'L1'` or `'Linf'`.

10.5 Dissipativity with a general linear supply: `mvst_pos_supply`

Dissipativity of a positive system is tested against a **general linear** supply rate (rather than a quadratic one),

$$\sigma(w, z) = q^\top z + r^\top w,$$

with weights q, r of arbitrary sign. A linear storage $V = \lambda^\top x$, $\lambda \geq 0$, certifies dissipativity through the copositive program

$$A^\top \lambda - C^\top q \leq 0, \qquad E^\top \lambda - F^\top q - r \leq 0$$

(with two further rows $(J^\top - I)\lambda - C_d^\top q_d \leq 0$ and $E_d^\top \lambda - F_d^\top q_d - r_d \leq 0$ when a jump channel is present). Choosing $q = -\mathbf{1}$, $r = \mathbf{1}$ recovers the mass-balance (passivity) supply $\sigma = \mathbf{1}^\top w - \mathbf{1}^\top z$; choosing $q = -\mathbf{1}$, $r = \gamma\mathbf{1}$ recovers the L_1-gain program.

The supply is given through its coefficients `qc` (on the output) and `rc` (on the disturbance). For the compartmental system the supply is satisfiable only once the disturbance weight `rc` clears the induced-gain threshold:

```
sys = struct('A',[-0.9 0;0.5 -0.6],'Ec',[1;0],'Cc',[1 1]);
r1 = mvst_pos_supply(sys, struct('supply',struct('qc',-1,'rc',1)));
r3 = mvst_pos_supply(sys, struct('supply',struct('qc',-1,'rc',3)));
fprintf('rc=1 feas=%d   rc=3 feas=%d\n', r1.feas, r3.feas);
```

```
rc=1 feas=0   rc=3 feas=1
```

At `rc = 1` the supply is not satisfiable; at `rc = 3` it is, consistent with the induced gain (2.037) lying between those two levels. When feasible, `res.lambda` is the storage weight.

10.6 Positivity-preserving stabilization: `mvst_pos_stabilize`

The goal is a state feedback $u = Kx$ that makes the closed loop **both** positive and stable. Positivity requires the closed-loop drift $A + B_c K$ to be Metzler, which is bilinear in $(K, \text{certificate})$. The copositive change of variables $X = \mathrm{diag}(\lambda)$ with $\lambda > 0$ and $U_c = KX$ linearizes it: $A + B_c K$ is Metzler iff $AX + B_c U_c$ has nonnegative off-diagonal entries. The design is then the linear program

$$(AX + B_c U_c)_{ij} \geq 0 \quad (i \neq j), \qquad (AX + B_c U_c)\mathbf{1} \leq -\varepsilon \mathbf{1},$$

in (λ, U_c), with $X = \mathrm{diag}(\lambda)$. The first rows force the closed loop Metzler (positivity preserved); the last is the copositive Lyapunov decrease $(A + B_c K)\lambda < 0$. The gain is recovered as $K = U_c X^{-1}$:

```
r = mvst_pos_stabilize(struct('A',[1 2; 0 1],'Bc',eye(2)), struct('class','ct'));
fprintf('feas=%d\n', r.feas); fprintf('K =\n'); disp(r.K);
```

```
feas=1
K =
   -1.0010   -2.0000
         0   -1.0010
```

The closed-loop drift $A + B_c K$ is Metzler with negative diagonal, hence positive and Hurwitz. When `opts.certify` is true (the default) the design is re-checked with `mvst_pos_stability` and returned in `res.cert`. A single shared input column cannot in general make an arbitrary A Metzler AND Hurwitz, so adequate actuation is required for feasibility. The impulsive class additionally designs a jump gain K_d so that $J + B_d K_d \geq 0$ under a constant dwell-time.

10.7 Worked case study: an epidemic-like positive system

We put the pieces together on a small compartmental model. Three coupled regions carry an infection; near the disease-free equilibrium the linearized prevalences $x = (x_1, x_2, x_3)$ obey $\dot{x} = Ax + Ew$ with

$$A = \begin{bmatrix} -1.6 & 0.5 & 0.2 \\ 0.4 & -1.8 & 0.6 \\ 0.3 & 0.4 & -2.0 \end{bmatrix}, \qquad E = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}, \qquad C = \begin{bmatrix} 1 & 1 & 1 \end{bmatrix}.$$

The off-diagonal entries (cross-region transmission and mobility) are nonnegative, so A is Metzler and the model is positive. The disturbance w is an imported-infection rate into region 1, and the output $z = x_1 + x_2 + x_3$ is the total prevalence.

Step 1: stability below the epidemic threshold. The copositive stability LP certifies the disease-free equilibrium:

```
Ae = [-1.6 0.5 0.2; 0.4 -1.8 0.6; 0.3 0.4 -2.0];
rs = mvst_pos_stability(struct('A',Ae), struct('class','ct'));
fprintf('stable=%d, margin=%.4f\n', rs.stable, rs.margin);
disp('copositive weight p='); disp(rs.p);
```

```
stable=1, margin=0.9000
copositive weight p=
     1
     1
     1
```

The uniform weight $p = \mathbf{1}$ works: $V(x) = x_1 + x_2 + x_3$ (the total prevalence itself) is a Lyapunov function, and $A^\top \mathbf{1} = (-0.9, -0.9, -1.2)^\top < 0$ gives the decrease margin 0.9.

Step 2: the L1 gain (total infection burden). The induced L_1 gain is the total infection-days produced per unit of imported infection:

```
rg = mvst_pos_gain(struct('A',Ae,'E',[1;0;0],'C',[1 1 1]), struct('norm','L1'));
fprintf('L1 gain = %.6f\n', rg.gamma);
```

```
L1 gain = 1.061947
```

Step 3: an above-threshold outbreak, stabilized with positivity preserved. Weaken recovery so the epidemic is now unstable, and design a per-region intervention $u = Kx$ (added removal, actuated by $B_c = I$) that returns the system to a positive, stable regime. A stronger decrease margin `eps` is asked for so the controlled outbreak decays visibly:

```
Au = [-0.3 0.5 0.2; 0.4 -0.4 0.6; 0.3 0.4 -0.5];
fprintf('open-loop eig = %s\n', mat2str(round(eig(Au)',4)));
```

```
rz = mvst_pos_stabilize(struct('A',Au,'Bc',eye(3)), struct('class','ct','eps',0.5));
disp('gain K='); disp(rz.K);
Acl = Au + rz.K;
fprintf('closed-loop Metzler offdiag min = %.4f\n', min(Acl(~eye(3))));
fprintf('closed-loop eig = %s\n', mat2str(round(eig(Acl)',4)));
fprintf('re-certified stable = %d\n', rz.cert.stable);
```

```
open-loop eig = [0.4098 -0.6699 -0.9398]
gain K=
   -0.2000   -0.5000   -0.2000
   -0.4000   -0.1000   -0.6000
   -0.3000         0   -0.4000
closed-loop eig = [-0.5 -0.9 -0.5]
re-certified stable = 1
```

The open loop has the unstable mode $+0.41$ (an ongoing outbreak). The designed gain yields the closed-loop drift

$$A_u + K = \begin{bmatrix} -0.5 & 0 & 0 \\ 0 & -0.5 & 0 \\ 0 & 0.4 & -0.9 \end{bmatrix},$$

which is Metzler (its off-diagonal minimum is 0) and Hurwitz (eigenvalues $-0.5, -0.9, -0.5$). Positivity is preserved and the outbreak is driven to extinction; `mvst_pos_stability` re-certifies the result.

10.8 Multi-mode systems and the object bridge

For a switched or Markov-jump positive system, pass a cell of per-mode drifts; the coupled Markov-jump program additionally reads the generator `Pi`. A multi-mode system carrying a common reset has no home in this family and is refused with `mvst:System:posUnsupported` rather than being silently misrouted. The internal LP wrapper is `mvst_pos_lp`, and the analysis front end that mirrors the book's example scripts is the `S.pos(what, opts)` sugar method on a `mvst.System`, which routes to the `mvst_pos_*` family.

10.9 Further positive-systems engines

The following engines complete the family. They are called directly, or reached through the routers `mvst_pos_gain`, `mvst_pos_stabilize`, and `mvst_pos_ms`, which dispatch to the timer-dependent (SOS) backends when `opts.dwell` is `'min'` or `'range'`.

10.9.1 `mvst_pos_oc`: linear-cost optimal control

`res = mvst_pos_oc(sys, opts)` solves the linear-cost optimal control of a positive system by the copositive costate programs: the value is linear, `V(x) = p'x`, and the optimal costate `p` solves a copositive program. `opts.mode` selects `'finite'` (horizon `opts.T`, jump times `opts.tk`, adjoint costate recursion), `'inf'` (LTI, bounded costate `p = -(A')^{-1} q`), or `'dwell'` (periodic dwell-time); `opts.q` is the nonnegative running cost weight and `opts.x0` the initial state.

```
r = mvst_pos_oc(struct('A',[-1 0;1 -2]),struct('mode','inf','q',[1;1],'x0',[1;1]));
fprintf('feas=%d Jopt=%.4f p0=%s\n',r.feas,r.Jopt,mat2str(r.p0.',4));
```

```
feas=1 Jopt=2.0000 p0=[1.5 0.5]
```

10.9.2 `mvst_pos_ms`: second-moment certificates for stochastic positive systems

`res = mvst_pos_ms(sys, opts)` certifies mean-square properties of positive systems driven by Brownian and Poisson noise, which the mean-level LP layer cannot see. `opts.what` is `'stability'` (default), `'tracegain'`, or `'peakgain'`, and `opts.cert` is `'spn'` (default, a split into a semidefinite and an entrywise-nonnegative part) or `'diag'` (diagonal quadratic certificate). The constant certificates are SOSTOOLS programs by default (any backend, including `'nosdp'`); the legacy YALMIP formulation is `opts.engine = 'yalmip'` (optional external dependency).

```
sys = struct('A0',[-1 0;1 -2],'Ac',sqrt(1.5)*eye(2));
r = mvst_pos_ms(sys,struct('what','stability','cert','diag'));
fprintf('feas=%d\n',r.feas);
```

```
feas=1
```

`res.feas` is the certificate flag, `res.D` the diagonal certificate (or `res.P` for `'spn'`). For this diagonal system the certificate exists for noise intensities up to a threshold computed in closed form (the test suite checks it at `s* = 1.6667` for `Ac = sqrt(s) I`).

10.9.3 `mvst_pos_ms_dwell`: timer-dependent second-moment certificates

`mvst_pos_ms_dwell(sys, what, cert, opts)` is the minimum / range dwell-time backend of `mvst_pos_ms`, with `opts.dwell`, `opts.Tmin`, `opts.Tmax`, and the polynomial degree `opts.deg` of the timer certificate. It is normally reached through `mvst_pos_ms`:

```
s = struct('A0',[-1 0.5;0.2 -1.5],'Ac',{{0.2*eye(2)}},'J',{{[0.6 0;0.1 0.5]}});
r = mvst_pos_ms(s,struct('what','stability','cert','diag','dwell','min','Tmin',0.5,'
    ↪ deg',2));
fprintf('feas=%d\n',r.feas);
```

```
feas=1
```

With `deg = 0` the timer certificate collapses to the constant (arbitrary dwell-time) certificate, so the verdict then equals that of the constant program; a higher degree can only enlarge the feasible set. It requires SOSTOOLS and an SDP backend.

10.9.4 `mvst_pos_gain_sos`: timer-dependent induced gains

`mvst_pos_gain_sos(sys, opts)` computes the `L_inf` x `ell_inf` (peak) and `L_1` x `ell_1` (trace) induced gains of a positive impulsive or switched system under a minimum or range dwell-time, by the copositive-SOS forms of the book's sliced conditions (forward reachability tube for the peak gain, backward adjoint for the trace gain). It is reached through `mvst_pos_gain`:

```
s = struct('A',[-1 0;1 -2],'Ec',[0.1;1.1],'Cc',[0 1],'Fc',0.3, ...
           'J',[0.1 1;0.1 0.1],'Ed',[0.3;0.3],'Cd',[0 1],'Fd',0.03);
r = mvst_pos_gain(s,struct('norm','Linf','class','impulsive','dwell','min','Tmin',1,
    ↪ 'deg',4));
fprintf('feas=%d gamma=%.4f\n',r.feas,r.gamma);
```

```
feas=1 gamma=1.1356
```

The gain is non-increasing in `Tmin` (a longer guaranteed dwell-time can only help).

10.9.5 `mvst_pos_stabilize_sos`: timer-dependent copositive synthesis

`mvst_pos_stabilize_sos(sys, opts)` designs a flow gain `Kc(tau)` and a jump gain `Kd` that make the closed loop both positive and exponentially stable under a constant, minimum, or range dwell-time, through the copositive change of variables of the book (so the conditions are linear at each timer value). With `opts.norm = 'Linf'` and disturbance/output data present it additionally minimizes the closed-loop induced `L_inf` gain. It is reached through `mvst_pos_stabilize`:

```
s = struct('A',[3 -1;2 -1],'Ec',[0.5;0.1],'Cc',[0 1],'Fc',0.1,'J',[2 1;0 1.2], ...
           'Ed',[0.8;0.3],'Cd',[0 1],'Fd',0.1,'Bc',[1;0],'Bd',[1;1]);
r = mvst_pos_stabilize(s,struct('class','impulsive','dwell','min','Tmin',0.2,'deg'
    ↪ ,2,'norm','Linf'));
fprintf('feas=%d gamma=%.4f\n',r.feas,r.gamma);
```

```
feas=1 gamma=0.5981
```

10.9.6 `mvst_sosmatrvar` and `mvst_sosposmatrvar`: matrix variables for SOSTOOLS programs

These two helpers declare polynomial matrix variables in a SOSTOOLS program and are used by the copositive-SOS engines above. `[prog,P] = mvst_sosmatrvar(prog, Z, [sp sq])` declares a free (sign-indefinite) `sp`-by-`sq` polynomial matrix in the monomial vector `Z`. `[prog,Q] = mvst_sosposmatrvar(prog, n, d, vars)` declares an `n`-by-`n` matrix of degree `d` that is positive semidefinite for every value of `vars` by construction (a scalar sum-of-squares polynomial when `n = 1`). Both are independent reimplementations, on the modern SOSTOOLS API, of helpers of M. Peet's MatrixTools; the attribution is in their help text.

```
syms t; prog = sosprogram(t);
[prog,Q] = mvst_sosposmatrvar(prog,2,2,t);              % 2x2 SOS matrix in t
[prog,w] = mvst_sosmatrvar(prog,monomials(t,0:2),[2 1]); % free 2x1 polynomial
    ↪ vector
```

Chapter 11

Further performance measures

This chapter covers the further performance measures (`mvst_perf`, method `S.perf`, and its dwell-time backend `mvst_perf_dwell`), which correspond to the **Further Performance Measures** chapter of the companion monograph. Every quantity is defined below as a display before the code that computes it, so the chapter stands on its own.

11.1 The performance measures

11.1.1 Signal norms

The performance measures are induced gains and averages built on the standard signal norms. For a signal s,

$$\|s\|_{L_2}^2 = \int_0^\infty \|s(t)\|^2\, dt \qquad \text{(energy)}, \qquad \|s\|_{L_\infty} = \sup_{t\geq 0} \|s(t)\| \qquad \text{(peak)},$$

with ℓ_2 and ℓ_∞ the discrete counterparts summed / maximized over the jump instants. The hybrid norms combine the flow (L) and jump (ℓ) parts of a mixed continuous-discrete signal.

11.1.2 The Gramian

The covariance measures are obtained from the **controllability (reachability) Gramian** Π, the state second moment under a unit-intensity disturbance w. For a Hurwitz continuous-time drift it is the steady solution of the Lyapunov equation

$$A\Pi + \Pi A^\top + B_w B_w^\top = 0,$$

and for the constant dwell-time impulsive system it is the periodic solution $\Pi(\tau) = e^{A\tau}\Pi_0 e^{A^\top \tau} + \int_0^\tau e^{As} B_w B_w^\top e^{A^\top s}\, ds$ with the reset $\Pi_0 = J\Pi(\bar{T})J^\top$ closed over one period. The output covariance is

$$S(\tau) = C\Pi(\tau)C^\top,$$

and the measures are its trace and its largest eigenvalue over a period.

11.1.3 The measures, defined

`S.perf(measure)` computes a named performance measure, routing per system class to the matching exact engine (Riccati / lifted-eigenvalue for the induced gains, Lyapunov Gramian for the covariance measures). The measures are the following.

The **mean-square L2 gain** `'l2'` is the worst-case energy amplification (the H_∞-type gain), delegated to `mvst_l2gain`:

$$\gamma_{L_2} = \sup_{w \neq 0} \frac{\|z\|_{L_2}}{\|w\|_{L_2}}.$$

The **trace gain** `'tracegain'` is its square,

$$\gamma_{\mathrm{tr}} = \gamma_{L_2}^2 = \sup_{w \neq 0} \frac{\|z\|_{L_2}^2}{\|w\|_{L_2}^2}.$$

The **energy-to-peak (generalized H2) gain** `'e2p'` is the induced $L_2 \to L_\infty$ gain,

$$\gamma_{\mathrm{e2p}} = \sup_{w \neq 0} \frac{\|z\|_{L_\infty}}{\|w\|_{L_2}} = \sqrt{\sup_{\tau} \lambda_{\max}\big(C\Pi(\tau)C^\top\big)}.$$

The **peak gain** `'peak'` is its square,

$$\gamma_{\mathrm{peak}} = \gamma_{\mathrm{e2p}}^2 = \sup_{\tau} \lambda_{\max}\big(C\Pi(\tau)C^\top\big).$$

The **H2 cost** `'h2'` is the average output power,

$$\gamma_{H_2} = \sqrt{\frac{1}{\bar{T}} \int_0^{\bar{T}} \mathrm{tr}\big(C\Pi(\tau)C^\top\big)\, d\tau},$$

which reduces to the classical H_2 norm in the jump-free case. The **deterministic peak-to-peak (star-norm)** `'p2p'` is the induced $L_\infty \to L_\infty$ gain; it is not covered by an exact engine (see below).

```
S = mvst.System.ct([-1 0.5; 0 -2], zeros(2,0), 'Bw', [1;0], 'C', [1 0]);
fprintf('h2 = %.6f\n', S.perf('h2').gamma);
fprintf('e2p = %.6f\n', S.perf('e2p').gamma);
fprintf('peak = %.6f\n', S.perf('peak').gamma);
```

```
h2 = 0.707107
e2p = 0.707107
peak = 0.500000
```

Note `peak` (0.5) is the square of `e2p` (0.7071), as defined. For a scalar output $C\Pi C^\top$ is a scalar, so its trace and largest eigenvalue coincide and `h2 = e2p`; with a vector output they differ. `res.gamma` carries the measure value and `res.measure` / `res.class` the resolved names.

11.1.4 Coverage matrix

Not every measure is characterized for every class by the exact engines. The dispatcher computes the pairs below and raises `mvst:perf:unsupported` (naming the nearest supported pair) for the rest:

measure	ct	mjls	switched	impulsive/sd	poisson
`l2`	yes	yes	yes	yes	yes
`tracegain`	yes	yes	yes	yes	yes
`e2p`	yes	no	no	yes	no
`peak`	yes	no	no	yes	no
`h2`	yes	no	no	yes	no

The induced gains (`l2`, `tracegain`) inherit every class of `mvst_l2gain`. The Gramian measures (`e2p`, `peak`, `h2`) are exact for the continuous-time system and its constant dwell-time

impulsive / sampled instance. Two further families are routed to the dwell-time backend `mvst_perf_dwell` (next section): the minimum and range dwell-time versions of `e2p`, `peak`, and `h2` (select with `opts.dwell`), and the deterministic peak-to-peak (star-norm) measure `p2p`, which needs an invariant-set LMI with a free decay rate. Both need an SOS or SDP backend. Pairs that are covered by none of these (for example the multi-mode stochastic covariance measures) are refused deliberately: rather than silently returning an approximate or wrong number, the dispatcher raises `mvst:perf:unsupported` and points at the closest supported measure.

11.2 Dwell-time and peak-to-peak measures

11.2.1 `mvst_perf_dwell`

`res = mvst_perf_dwell(sys, measure, opts)` is the backend of `mvst_perf` for the timer-dependent measures; it is normally reached through `S.perf` or `mvst_perf`. The measure is `'peak'`, `'e2p'`, or `'h2'` with `opts.dwell` in {`'min'`,`'range'`} (and `opts.Tmin`, `opts.Tmax`, `opts.deg` for the degree of the timer certificate), or `'p2p'` / `'peakfull'` for the deterministic peak-to-peak and amplitude-bounded peak gains. The dwell-time tubes and, by default, the decay-rate certificates (a two-parameter bisection) use SOSTOOLS; the legacy YALMIP formulation of the decay-rate certificates is `opts.engine = 'yalmip'` (optional external dependency, not bundled).

```
s = struct('A',[-0.2 1;-1 -0.2],'Bw',[0.5;0.3],'C',[1 0],'J',0.6*eye(2));
r = mvst_perf(s,'h2',struct('dwell','min','Tmin',1,'deg',4));
fprintf('h2 = %.4f\n',r.gamma);
```

```
h2 = 0.7105
```

The same system under the deterministic peak-to-peak measure:

```
r = mvst_perf(s,'p2p',struct('dwell','min','Tmin',1,'deg',4));
fprintf('p2p = %.4f (lower bound field gammaLB)\n',r.gamma);
```

```
p2p = 2.1523 (lower bound field gammaLB)
```

`res.gamma` is the certified value (an upper bound for `p2p`, with the DC-gain companion lower bound in `res.gammaLB`), and `res.lambda`, `res.nu` the decay parameters at the optimum. The `p2p` call takes noticeably longer than the others because of the bisection.

11.3 Worked case study: a performance sweep

We take a lightly damped second-order mode (a resonant flexible mode) with natural frequency $\omega_0 = 1$ and damping ratio $\zeta = 0.15$,

$$A = \begin{bmatrix} 0 & 1 \\ -\omega_0^2 & -2\zeta\omega_0 \end{bmatrix}, \qquad B_w = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \qquad C = \begin{bmatrix} 1 & 0 \end{bmatrix},$$

with the disturbance entering the velocity channel, measure its performance, and sweep the damping.

Step 1: measure the performance.

```
w0 = 1.0; zeta = 0.15;
A = [0 1; -w0^2 -2*zeta*w0]; Bw = [0;1]; C = [1 0];
S = mvst.System.ct(A, zeros(2,0), 'Bw', Bw, 'C', C);
fprintf('h2   = %.6f\n', S.perf('h2').gamma);
fprintf('e2p  = %.6f\n', S.perf('e2p').gamma);
```

```
fprintf('peak = %.6f\n', S.perf('peak').gamma);
fprintf('l2   = %.6f\n', S.perf('l2').gamma);
```

```
h2   = 1.290994
e2p  = 1.290994
peak = 1.666667
l2   = 3.371521
```

Step 2: sweep the damping ratio. As ζ falls, the resonance sharpens and every gain grows; the L_2 (H_∞) gain, which sees the resonant peak directly, grows fastest:

```
fprintf('%6s %10s %10s %10s\n','zeta','h2','e2p','l2');
for z = [0.10 0.15 0.25 0.40 0.70]
    Az = [0 1; -w0^2 -2*z*w0];
    Sz = mvst.System.ct(Az, zeros(2,0), 'Bw', Bw, 'C', C);
    fprintf('%6.2f %10.6f %10.6f %10.6f\n', z, ...
            Sz.perf('h2').gamma, Sz.perf('e2p').gamma, Sz.perf('l2').gamma);
end
```

```
  zeta         h2        e2p         l2
  0.10   1.581139   1.581139   5.025208
  0.15   1.290994   1.290994   3.371521
  0.25   1.000000   1.000000   2.065613
  0.40   0.790569   0.790569   1.363892
  0.70   0.597614   0.597614   1.000245
```

The H_2 and energy-to-peak costs coincide because the output is scalar (see the note on scalar outputs above), while the L_2 gain climbs from 1.0 at $\zeta = 0.7$ to 5.0 at $\zeta = 0.1$: a lightly damped mode is far more sensitive to worst-case disturbance energy than the average-power measures suggest.

The exact engines used in this case study need only base MATLAB; the dwell-time backend `mvst_perf_dwell` needs SOSTOOLS and an SDP backend.

Chapter 12

Book reproduction

This chapter explains how the toolbox regenerates every figure and every printed number of the companion monograph, where the outputs are written, and (for the development tree) how a golden regression harness freezes those numbers.

12.1 The figure and data pipeline

The reproduction layer rests on one idea: the numbers the book prints and the numbers the code computes are the **same** numbers, carried across by generated files. There are three stages.

1. An **example script** (`examples/ex_*.m`, `examples/fig_*.m`, or `examples/comp_*.m`) sets up a system, calls the analysis or synthesis engines, and produces two kinds of artifact.
2. Every plotted figure is exported as a vector PDF to the output folder.
3. Every scalar, matrix, or string the book quotes in its running text is written as a LaTeX macro to the `data/` subfolder of the output folder.

The output folder is returned by `mvst_figdir`, which is the same for every example. In order of precedence it is the environment variable `MVST_FIGDIR` when set and nonempty, and otherwise `./figures` under the current folder. The folder and its `data/` subfolder are created when missing, so the examples run standalone from any clone:

```
setenv('MVST_FIGDIR', fullfile(tempdir, 'mvst_out'));
mvst_figdir('data')        % -> .../mvst_out/data
```

The book manuscript then `\inputs` the `data/*.tex` files and refers to each quantity by macro name. A sentence such as "the critical level is $\gamma^\star = [\text{value}]$" resolves, at LaTeX compile time, to the value the MATLAB run wrote. No number is transcribed by hand, so the text cannot drift away from the code.

Schematically, for one example:

```
ex_game_sd.m  --->  <figdir>/ex_game_sd.pdf          (a figure in the book)
              --->  <figdir>/data/game_sd.tex        (\def\GameGammaSd{...})
                          |
                          v
              Book_Springer.tex \input{data/game_sd.tex}
                          |
                          v
              "...gamma* = \GameGammaSd ..."   (typeset value)
```

The data macros are written through `mvst_texdata`, which opens a `.tex` file and exposes `.num(name,value)` for a formatted scalar, `.mat(name,M)` for a LaTeX `bmatrix`, and `.str(name,text)` for a string, then `.close()`. Matrices are formatted by `mvst_fmtmat`. This writer is the single choke point for everything the book quotes.

12.2 Figures and the export layer

The figure and export layer lives in its own folder, `bookutil/`, separate from the engine layer `lib/`: nothing in the analysis engines depends on it, so the solvers can be used headless without ever loading a plotting routine. `mvst_setup` adds `bookutil/` to the path; only `mvst_figdir` (the output folder, see above) stays in `lib/`.

- `mvst_gemcolors`, `mvst_latexdefaults`, and `mvst_figstyle` set the MATLAB "gem" color order, route all text through the LaTeX interpreter, and force a white background with black text before export;
- `mvst_savepdf` applies the white style and writes a cropped vector PDF, so the exports are correct even when MATLAB itself is in dark mode;
- `mvst_texdata` and `mvst_fmtmat` write the data macros;
- `mvst_stabfig`, `mvst_stabtable`, `mvst_steer_components`, and `mvst_steer_ellipse` draw the recurring chapter figures and generated table rows.

12.3 The runners

Each book chapter has a per-chapter runner at the top level of `matlab/`. A runner executes its chapter's example scripts with a fresh workspace (through `mvst_runscripts`), writing the PDFs and data macros described above.

Runner	Regenerates (examples)	Needs SOS solver	Rough runtime
`run_stability`	dwell-scalar, Levy, rate-SOS, sampled-data-SOS, stabz, switched-SOS	yes (the `*_sos`)	minutes
`run_stabilization`	the `ex_stabz_*` family (CT, DT, delay, MJLS, sampled-data, NCS, ...)	one SOS script	tens of seconds
`run_oc`	time-varying MJLS / sampled / switched costs and trajectories, the four horizon figures	no	tens of seconds
`run_steering`	covariance steering for MJLS, Poisson, sampled-data, switched	no	seconds
`run_dissipativity`	dissipativity for MJLS, Poisson, sampled-data, switched (plus time-varying)	no	tens of seconds
`run_game`	H-infinity game level for the five classes plus the gamma iteration	no	tens of seconds

Runner	Regenerates (examples)	Needs SOS solver	Rough runtime
`run_computational`	ARE, MJLS, and the SOS optimal-control / dissipativity / game examples	yes	minutes
`run_vdiss`	vector-dissipativity example	no	seconds

The runtimes are indicative: the base-MATLAB runners finish in seconds to a minute, while the SOS runs (`run_computational` and the `*_sos` scripts) are dominated by the external SDP solver and take several minutes.

`run_all` is the thin wrapper that calls every runner plus the remaining standalone example scripts (the cones, positive-systems, and performance examples):

```
run_all
```

The SOS / SDP examples require an SDP solver on the path (see Getting started); the rest need only base MATLAB. To regenerate just one chapter, call its runner, for example `run_dissipativity`. Build the book first if you will typeset it: the data macros are `\input` by the manuscript, and some example scripts cross-reference the built document.

12.4 The golden regression harness (development tree only)

The development tree carries a golden-data regression harness tied to the book's figure data. It is not part of the release battery (the release battery is the 26 unit-test suites run by `run_all_tests`, see Getting started), and it is left out of the release tree.

The idea is mechanical. The book quotes hundreds of computed numbers, and a refactor of the numerical core must not change any of them. When the code was known good, the whole data-macro corpus was captured into `tests/golden/data/`. `tests/mvst_regress` re-parses every golden file and the live `data/*.tex` files and compares them macro by macro with numeric tolerance (default relative `1e-6`, absolute `1e-12`); on any drift it raises `mvst:regress:failed`. `tests/run_regress` is the strict `-batch` wrapper. Drift can be inspected without failing through report mode:

```
rep = mvst_regress('Report', true);
rep.files(find(~[rep.files.pass], 1)).diffs   % the first failing file's diffs
```

A change that is intended to alter a printed number requires re-baselining on purpose: regenerate the live data, review the report so that only the expected macros moved, re-capture the snapshot with a refreshed `tests/golden/MANIFEST.md`, and commit the new golden files together with the code change that caused them.

Chapter 13

API reference

Every public function of the toolbox, grouped by area, with the first line of its help (`help <name>` for the full text). The table below is generated directly from the source files, so it stays accurate. A few summaries are the first physical help line and read as sentence fragments; the continuation is in the function's own help.

The recommended entry point is the `+mvst` package (`mvst.System`, `mvst.Options`, `mvst.solvercheck`); every `mvst_*` engine remains callable directly.

13.1 How to read this reference

The tables group the toolbox by area, in the order the manual map walks the material. Each row names a public function and the first line of its help. To use a function you need three things, all reachable from here:

- **What it takes.** Call `help <name>` for the full input contract. Systems are built through the `mvst.System` factories (see Systems); the low-level engines also accept the thin/rich structs described in `lib/MODEL_CONVENTION.md`.
- **What it returns.** Every analysis or synthesis routine returns a single result struct. The recurring field names follow the conventions in Result-struct conventions below, so once you know the pattern you can read any engine's output.
- **How it can fail.** Errors carry a structured identifier in the `mvst:...` namespace (Error-id namespace); catch them by id rather than by message text.

The two-layer surface is intentional. The object layer (`S.stability()`, `S.l2gain()`, `S.perf(...)`) is the release-facing API and is what the manual uses; the `mvst_*` engines are the computational core the object methods dispatch to, and stay callable directly for scripting and for the book's example files. Where a name below reads as a sentence fragment, it is the first physical help line; the continuation is in the function's own help.

13.1.1 Object API (`+mvst` package)

Function	Summary
`mvst.System`	Unified value-type wrapper for a matrix-valued dynamical system.
`mvst.Options`	Solver options for the MVST package.
`mvst.solvercheck`	Detect and probe the SDP / SOS toolchain on the MATLAB path.

13.1.2 Model builders and bridges

Function	Summary
mvst_mode	Build a (normalized) controlled-mode data structure.
mvst_mode_tv	Build a TIME-VARYING controlled-mode data structure.
mvst_system	Assemble a (possibly multi-mode) system for the solvers.
mvst_torich	Convert a thin mode/system struct to the rich SOS-layer struct.
mvst_tothin	Convert a rich SOS-layer struct back to a thin mode struct.
mvst_getf	Field of struct S named F, or default D if absent/empty.
mvst_evalc	Evaluate a (possibly time-varying) coefficient at time t.

13.1.3 Symmetric-subspace algebra

Function	Summary
mvst_svecbasis	Orthonormal basis matrix of the symmetric subspace.
mvst_liftsym	Reduced matrix representation of X -> He(A0 X) + sum C_l X C_l'.
mvst_vech	Stack the independent (upper-triangular) entries of symmetric M.
mvst_unvech	Rebuild a symmetric n-by-n matrix from MVST_VECH output.

13.1.4 Stability and stabilization

Function	Summary
mvst_stability	Exact mean-square stability analysis for the book's system classes.
mvst_stabilize	Exact state-feedback stabilization for the book's system classes.
mvst_stab_sos	Generic SOSTOOLS stability-ANALYSIS test for linear impulsive
mvst_stab_sos_modes	Generic SOSTOOLS stability-ANALYSIS test on a PRODUCT CONE
mvst_stabz_sos	Generic SOSTOOLS state-feedback stabilization synthesis for
mvst_stabz_sd_range	Aperiodic (range) sampled-data refresh-gain synthesis for a held-input matrix-valued plant, as a matrix sum-of-squares program in the timer.

Function	Summary
mvst_stab_critical	Critical parameter value on the mean-square stability margin.
mvst_stab_delay	Delay-independent stability of matrix-valued (and positive) time-delay systems (Chapter "Stability", Section sec:stab:delay, and Chapter "Linear Positive Systems", prop:pos:classncs).

13.1.5 Optimal control (Riccati and SOS)

Function	Summary
mvst_solve_finite	Finite-horizon linear-cost optimal control (CT).
mvst_solve_infinite	Infinite-horizon linear-cost optimal control (CT).
mvst_solve_average	Long-term average-cost optimal control (CT).
mvst_solve_finite_tv	Finite-horizon linear-cost optimal control, TIME-VARYING.
mvst_solve_infinite_tv	Infinite-horizon dual variable, TIME-VARYING.
mvst_ric_ct	Continuous-time I-GRE residual and optimal gain for one mode.
mvst_ric_ct_tv	Time-varying continuous-time I-GRE residual and gain.
mvst_dre_ct	Backward solution of the continuous-time differential I-GRE.
mvst_dre_ct_tv	Backward time-varying differential I-GRE.
mvst_dre_ct_poisson	Backward LQ Riccati for GBM + Poissonian-jump systems.
mvst_care_ct	Infinite-horizon CT I-GRE (stochastic ARE) for one mode.
mvst_care_ct_poisson	Algebraic generalized Riccati for GBM+Poisson systems.
mvst_igre_periodic_sd	Periodic-sampling sampled-data optimal control.
mvst_oc_sos	Optimal control by the dual (storage-function) SOS/LMI program
mvst_oc_sos_modes	Optimal control by the dual (storage-function) SOS/LMI
mvst_oc_dwell	Causal minimum / range dwell-time optimal control of a single-cone linear impulsive system (Chapter "Linear Cost", Section sec:OC:dwell): the timer-dependent forward-Riccati / jump-GRE programs Theorem th:OC:MDT (minimum dwell-time) and Theorem th:OC:RDT (range dwell-time), with the average-cost bounds th:IMP:AC:dwell:MDT / th:IMP:AC:dwell:RDT as a secondary mode.

Function	Summary
mvst_odecap	Terminal ODE event: stop when the state norm reaches CAP.

13.1.6 Steering

Function	Summary
mvst_steer	Class-dispatched finite-time optimal covariance steering.
mvst_steer_newton	Damped Newton for the steering multiplier.
mvst_steer_components	Covariance entries versus time.
mvst_steer_ellipse	Single-panel covariance-steering figure.

13.1.7 Dissipativity, L2-gain, and games

Function	Summary
mvst_l2gain	Class-dispatched L2-gain (thin router over the exact backends).
mvst_l2gain_ct	Continuous-time L2-gain (H-infinity norm) by bisection.
mvst_l2gain_ct_mjls	Exact coupled mean-square L2-gain of an MJLS.
mvst_l2gain_dt	Discrete-time L2-gain (H-infinity norm) by exact bisection.
mvst_l2gain_dt_mjls	Coupled mean-square l2-gain of a discrete-time MJLS.
mvst_l2gain_ct_dwell	Dwell-time-certified L2-gain bound, switched system.
mvst_l2gain_ct_poisson	Mean-square L2-gain of a GBM+Poisson system.
mvst_l2gain_sd_hybrid	Exact hybrid L2-gain, flow disturbance / sampled output.
mvst_diss_sos	Generic SOSTOOLS dissipativity-ANALYSIS program for linear
mvst_diss_sos_modes	Generic SOSTOOLS dissipativity-ANALYSIS program on a
mvst_diss_sos_selftest	Validation of MVST_DISS_SOS against exact ground truth.
mvst_diss_dwell_cert	MDT dissipativity certificate for a switched system.
mvst_diss_pT_ct	Finite-horizon storage matrix versus horizon length.
mvst_passivity_ct	Passivity test of a square LTI system by a frequency sweep.
mvst_passivity	Routed passivity analysis with indices (exact CT test by default).
mvst_hinfnorm	H-infinity norm / (mean-square) L2-gain with a uniform result.

Function	Summary
mvst_isp_margin_ct	Input-strict-passivity index of a (stochastic) LTI flow.
mvst_kyp	Operator-domain Kalman-Yakubovich-Popov engine for matrix-valued dissipativity (Chapter "Dissipativity", Section sec:diss:kyp): the general KYP lemma for an arbitrary-sign supply matrix M, in continuous time, discrete time, and the hybrid (periodic impulsive) setting.
mvst_qsr_supply	Validate and unpack a quadratic (Q,S,R) supply contract.
mvst_dissyn_sos	Generic SOSTOOLS dissipative state-feedback SYNTHESIS program
mvst_dissyn_sos_selftest	Validation of MVST_DISSYN_SOS by closed-loop
mvst_dof_sos	Generic SOSTOOLS timer-dependent output-feedback bracket
mvst_game_gamma	Class-dispatched H-infinity game level (thin router).
mvst_game_gamma_ct	Critical disturbance level gamma* of the H-infinity game.
mvst_game_gamma_ct_mjls	Exact coupled critical level of the MJLS game.
mvst_game_gamma_ct_dwell	Certified achievable H-infinity level, switched.
mvst_game_gamma_ct_poisson	Critical level gamma* for GBM+Poisson games.
mvst_game_dre_ct	Backward game (HJI) differential Riccati, continuous time.
mvst_game_dre_ct_poisson	Backward game (HJI) Riccati for GBM+Poisson systems.
mvst_game_average	Infinite-horizon average (ergodic) zero-sum game value (Chapter "Game", th:game:AC:finite / th:game:AC:suff, cor:game:AC:equiv).
mvst_game_mixed	Infinite-horizon mixed H2/H-infinity design as a two-player nonzero-sum Nash game (Chapter "Game", prop:game:mixed:verify, eq:game:mixed:GRE).
mvst_game_nash	Infinite-horizon N-player Nash equilibrium via the coupled I-GRE system (Chapter "Game", prop:game:nash:verify, eq:game:nash:GRE).
mvst_game_sd_held	Periodic HJI-GRE of the held-input sampled-data game.
mvst_game_gamma_sd_held	Critical level of the held-input sampled-data game.
mvst_game_sos	Generic SOSTOOLS state-feedback zero-sum GAME / minimax program
mvst_game_sos_selftest	Validation of MVST_GAME_SOS against exact ground truth.

13.1.8 Further performance measures

Function	Summary
`mvst_perf`	Generic performance-analysis dispatcher (Chapter "Further
`mvst_perf_dwell`	Dwell-time and deterministic-peak backends for MVST_PERF (Chapter "Further Performance Measures").

13.1.9 Positive systems (copositive LP)

Function	Summary
`mvst_pos_stability`	Copositive linear-programming stability analysis for
`mvst_pos_gain`	Induced-gain copositive linear programs for LTI positive
`mvst_pos_stabilize`	Positivity-preserving state-feedback stabilization by
`mvst_pos_supply`	General-linear-supply dissipativity by copositive linear
`mvst_pos_lp`	Internal LINPROG wrapper for the positive-systems LP family.
`mvst_pos_gain_sos`	Timer-dependent copositive-SOS induced-gain programs for positive impulsive / switched systems (minimum and range dwell-time), the timer-dependent branch of MVST_POS_GAIN.
`mvst_pos_ms`	Second-moment (mean-square) certificates for STOCHASTIC positive systems (Chapter "Linear Positive Systems", stochastic layer): the covariance-level SDP certificates that see the Brownian and Poisson noise, which the mean-level copositive-LP layer (MVST_POS_STABILITY / _GAIN) is blind to.
`mvst_pos_ms_dwell`	Timer-dependent (minimum / range dwell-time) second-moment certificates for stochastic positive systems, the SOS-timer backend of MVST_POS_MS.
`mvst_pos_oc`	Linear-cost optimal control of linear positive systems by the copositive costate programs of Chapter "Linear Positive Systems" (Section sec:pos:oc).
`mvst_pos_stabilize_sos`	Timer-dependent copositive-SOS state-feedback synthesis for positive impulsive systems under constant / minimum / range dwell-time, the timer-dependent branch of MVST_POS_STABILIZE.

13.1.10 Simulation

Function	Summary
mvst_sample_ct	Euler--Maruyama sample paths of a controlled diffusion.
mvst_sample_mjls	Sample paths of a Markov-jump controlled diffusion.
mvst_simulate_ct	Closed-loop second-moment (covariance) simulation (CT).
mvst_simulate_mjls	Closed-loop second-moment simulation for an MJLS.
mvst_simulate_impulsive	Closed-loop covariance of an impulsive system.

13.1.11 Solver dispatch

Function	Summary
mvst_sossolve	Solve a SOSTOOLS program with the backend selected by OPTS.SOLVER.
mvst_sosmatrvar	Free (sign-indefinite) polynomial matrix variable.
mvst_sosposmatrvar	SOS (positive-semidefinite) polynomial matrix variable.
mvst_opts	Normalize solver options and verify the SDP/SOS solver is present.

13.1.12 Figures and export

mvst_figdir (in lib/) returns the output folder of the examples. The other helpers live in bookutil/ and are used by the example scripts only.

Function	Summary
mvst_figdir	Output folder of the example scripts (figures and data macros).
mvst_gemcolors	MATLAB's built-in "gem" color order (with a fallback).
mvst_latexdefaults	Use the LaTeX interpreter for all new text objects.
mvst_figstyle	Force a white background and black text on a figure.
mvst_savepdf	Apply the white style and export a figure to a vector PDF.
mvst_texdata	Writer for LaTeX data macros (gains, costs, systems).
mvst_fmtmat	Format a numeric matrix as a LaTeX bmatrix string.
mvst_stabfig	Standard open-vs-closed stabilization figures.
mvst_stabtable	Emit one row of the Stabilization-chapter summary table.
mvst_runscripts	Run a list of example scripts with a fresh workspace each.

13.2 Result-struct conventions

Every engine returns one struct. The field names are consistent across the toolbox, so the same conventions apply everywhere. The recurring fields are:

- `res.feas` (logical): the program was feasible / a certificate was found. Used by the synthesis and copositive-LP routines (`mvst_pos_*`, `mvst_stabilize`, `mvst_dissyn_sos`, ...). When `res.feas` is false the solution fields (`res.K`, `res.p`, ...) are empty rather than garbage.
- `res.stable` (logical): the analysis certificate holds. Returned by the stability analyses (`mvst_stability`, `mvst_pos_stability`).
- `res.gamma` / `res.gammastar`: the computed level. `res.gamma` is a gain or cost (L2-gain, induced positive-system gain, performance measure); `res.gammastar` is the critical game level (`mvst_game_gamma`).
- `res.K` (and `res.Kd` for a jump gain): the synthesized state-feedback gain. Synthesis routines also return the certificate in `res.P` (a matrix storage / Lyapunov certificate) or `res.lambda` / `res.p` (a copositive weight).
- `res.margin`: the achieved strict-inequality slack of a certificate (the smallest row slack; positive certifies the strict inequalities).
- `res.cost`: the optimal value of an optimal-control problem (`mvst_solve_*`), with the associated `res.P` / `res.P0` matrices.
- `res.class` / `res.method` / `res.measure`: the resolved dispatch tag (which system class or which measure was actually computed), useful when the class was inferred rather than passed explicitly.
- `res.conv` / `res.residual` / `res.iters`: convergence diagnostics for the iterative and flow-based engines (`mvst_solve_infinite`, `mvst_steer`).

Trajectory-valued engines add time-series fields (`res.t`, `res.Sigma`, `res.Pi`, `res.trSigma`, `res.curve`, `res.spectrum`) alongside the scalar summary. Some synthesis routines also return a `res.cert` / `res.closedloop` / `res.check` sub-struct that re-certifies the designed closed loop.

13.3 Error-id namespace

All errors raised by the toolbox carry a structured identifier of the form `mvst:<area>:<reason>`; catch them by identifier, not by message text. The `<area>` segment mirrors the function family:

Area prefix	Raised by	Typical reason
`mvst:System:`	the `mvst.System` factories	dimension mismatch, bad generator
`mvst:opts:` / `mvst:mode:`	option and model builders	missing / malformed field
`mvst:pos:`	the positive-systems LP family	missing data, bad class/norm, no Optimization Toolbox
`mvst:perf:`	`mvst_perf`	unsupported class-measure pair, missing I/O
`mvst:stability:` / `mvst:stabilize:`	stability / stabilization	bad class, coupling, dwell-time data
`mvst:l2gain:` / `mvst:game_gamma:`	gain / game routers	missing per-class data
`mvst:steer:`	steering	bad target, horizon, or class

Area prefix	Raised by	Typical reason
`mvst:tests:`	`run_all_tests`	a test truly failed (skips are not failures)

Two identifiers are worth knowing specifically: `mvst:pos:noOptimToolbox` (the positive-systems LPs need `linprog`) and `mvst:perf:unsupported` (a performance measure the book does not characterize for that class; the message names the nearest supported pair).

13.4 Troubleshooting

A solver is missing. The SOS layer (`mvst_*_sos`) needs SOSTOOLS and an SDP backend (SeDuMi / SDPT3 / MOSEK); the positive-systems LPs need the Optimization Toolbox. Run `mvst.solvercheck` to see what resolves and actually solves on your path. `mvst_setup` adds the bundled solvers of `thirdparty/`; SeDuMi and SDPT3 may need `install_sedumi` / `install_sdpt3` once, and `mvst.Options('solver','nosdp')` runs the SOSTOOLS-based engines without any SDP solver (see Getting started). A missing LP solver surfaces as `mvst:pos:noOptimToolbox`; a missing SDP backend surfaces from the SOS routine's solver call.

An analysis returns `stable=0` or a synthesis returns `feas=0`. This is a result, not an error: the certificate does not exist for the given data. Check that the model matches the class (a Metzler drift for the positive family, a Hurwitz drift for the continuous-time Gramian measures), and that any dwell-time is on the feasible side of its threshold. The dwell-time routines expose `opts.threshold` to bisect the feasibility boundary and return it in `res.Tstar`, which tells you how far the data is from feasibility.

Tolerance knobs. The copositive LPs take `opts.margin` (the strict-inequality slack) and `opts.pmin` (the lower bound normalizing the positive certificate); `mvst_pos_stabilize` takes `opts.eps` (the copositive decrease margin, raise it for a faster closed-loop decay). The Gramian engines take ODE tolerances (`opts.reltol`, `opts.abstol`) and quadrature counts (`opts.npts`, `opts.profpts`).

An `mvst.System` call errors on construction. The factories validate dimensions eagerly and raise `mvst:System:dim` (or `:generator`) with the offending block named; fix the block sizes rather than the call site. `help mvst.System.<factory>` lists the expected shapes.